第3章 流水线技术

张晨曦 王冬青

3.1 流水线的基本概念
3.2 流水线的性能指标
3.3 流水线的相关与冲突
3.4 流水线的实现
3.5 向量处理机

3.1 流水线的基本概念

3.1.1 什么是流水线

1. 工业生产流水线

下面通过一个例子来说明流水线的好处:

- 两种方案：无流水线（如手工作坊）和流水线（如工厂）
- 两种方案的工作过程对比@1
- 流水线生产过程的抽象描述@2
- 这种流水工作方式的主要特点

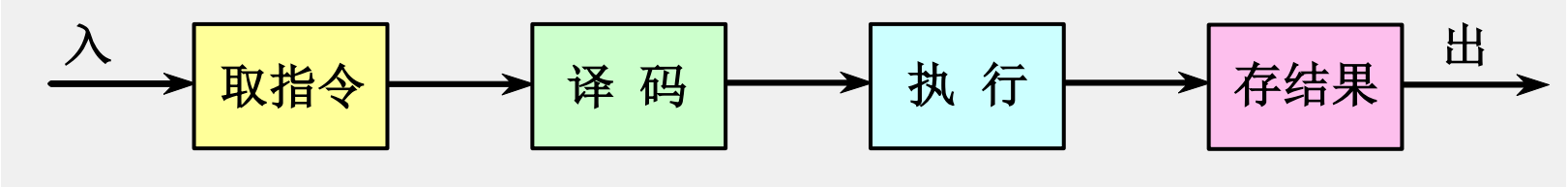
2. 流水线技术（pipelining）
   - 把一个重复的过程分解为若干个子过程，每个子过程由专门的功能部件来实现。
   - 把多个处理过程在时间上错开，依次通过各功能段，这样，每个子过程就可以与其他的子过程并行进行。
3. 流水线中的每个子过程及其功能部件称为流水线的级或段，段与段相互连接形成流水线。流水线的段数称为流水线的深度（pipeline depth）。

4. 指令流水线

- 把指令的解释过程分解为分析和执行两个子过程，并让这两个子过程分别用独立的分析部件和执行部件来实现。

  理想情况：速度提高一倍

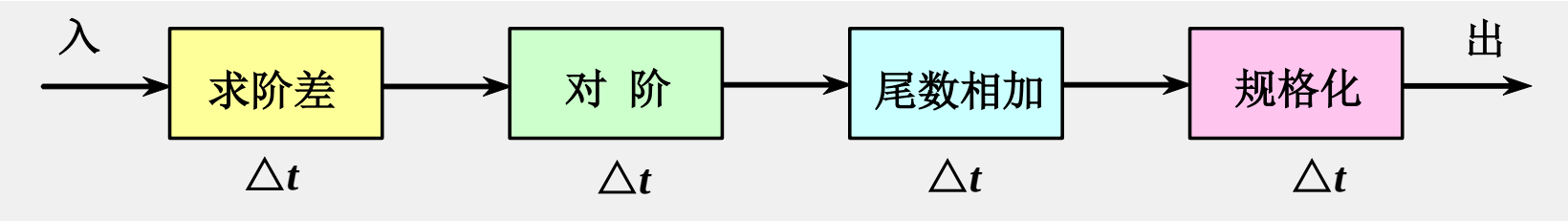
- 4段指令流水线



5. 浮点加法流水线@3

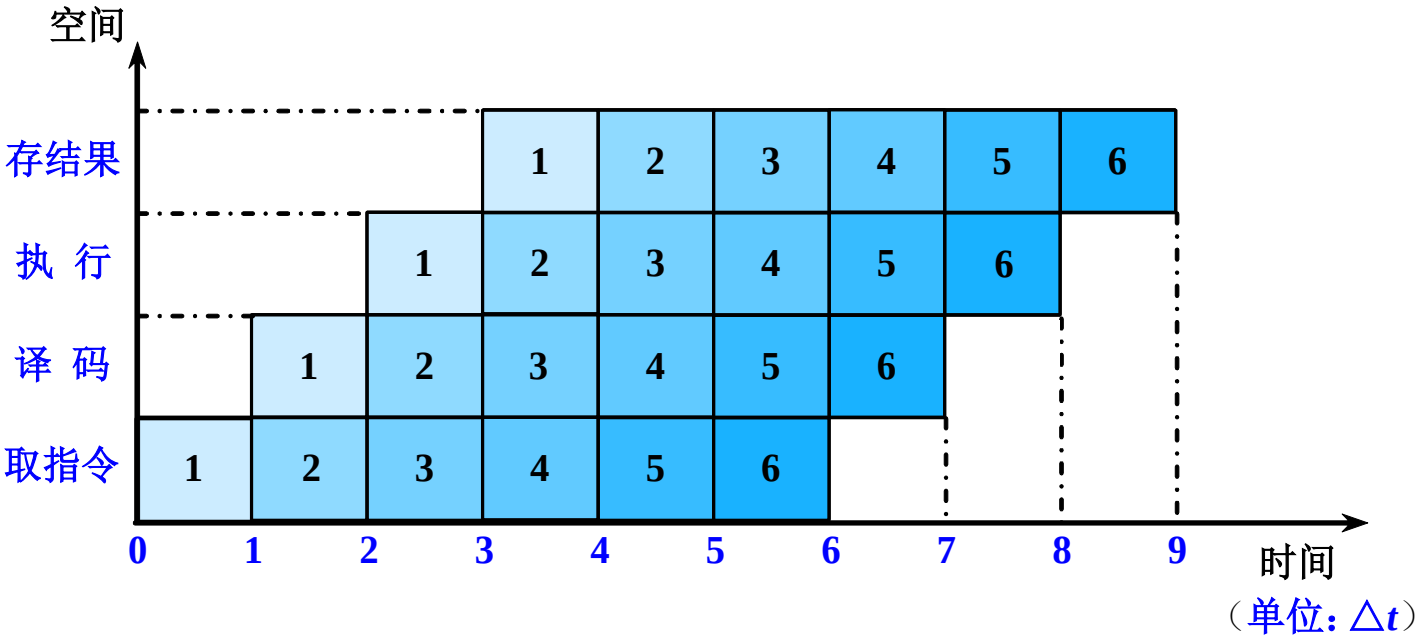
- 把流水线技术应用于运算的执行过程，就形成了运算操作流水线，也称为部件级流水线。
- 把浮点加法的全过程分解为求阶差、对阶、尾数相加、规格化4个子过程。

  理想情况：速度提高3倍



6. 时空图@4

- 时空图从时间和空间两个方面描述了流水线的工作过程。时空图中，横坐标代表时间，纵坐标代表流水线的各个段。
- 4段指令流水线的时空图



7. 流水技术的特点

① 合作：流水线把一个处理过程分解为若干个子过程（段），每个子过程由一个专门的功能部件来实现。

② 瓶颈：流水线中各段的时间应尽可能相等，否则将引起流水线堵塞、断流。

时间长的段将成为流水线的瓶颈。

③ 流水寄存器：流水线各功能部件间都要有一个缓冲寄存器（锁存器）。

- 作用：在相邻的两段之间传送数据，以保证提供后面要用到的数据，并把各段的处理工作相互隔离。

④ 流水技术适合于大量重复的时序过程，只有在输入端不断地提供任务，才能充分发挥流水线的效率。

⑤ 流水线需要有通过时间和排空时间@5。

- 通过时间：第一个任务从进入流水线到流出结果所需的时间。
- 排空时间：最后一个任务从进入流水线到流出结果所需的时间。

两个时间段流水线均不满载

3.1.2 流水线的分类

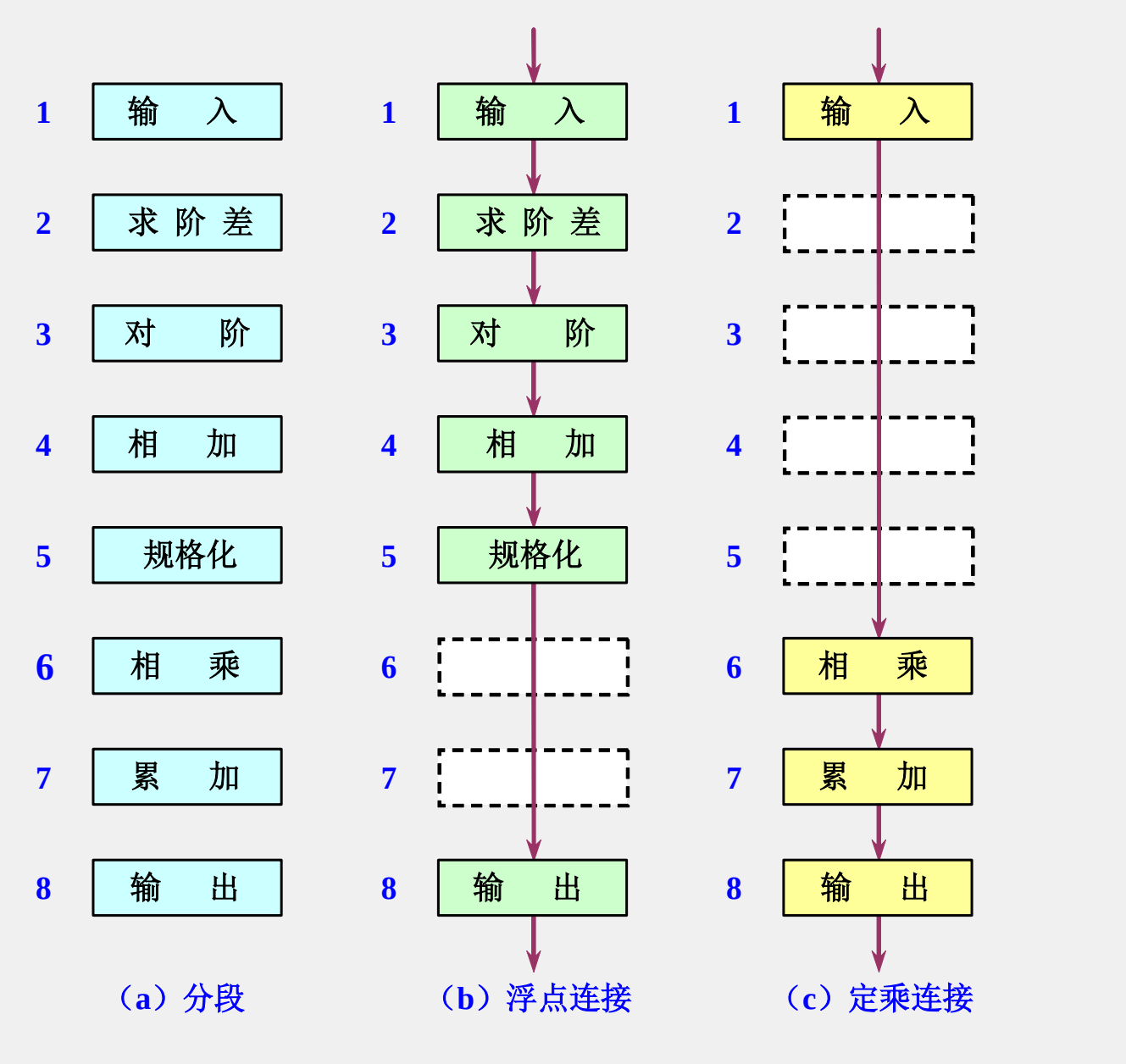
从不同的角度和观点，把流水线分成多种不同的种类。

1. 单功能流水线与多功能流水线

（按照流水线所完成的功能来分类）

- 单功能流水线：只能完成一种固定功能的流水线。
- 多功能流水线：流水线的各段可以进行不同的连接，以实现不同的功能。

例：ASC处理机的多功能流水线@6


1 输 入
2 求 阶 差
3 对 阶
4 相 加
5 规格化
6 相 乘
7 累 加
8 输 出
（a）分段
1 输 入
2 求 阶 差
3 对 阶
4 相 加
5 规格化
6
7
8 输 出
（b）浮点连接
1 输 入
2
3
4
5
6 相 乘
7 累 加
8 输 出
（c）定乘连接

2. 静态流水线与动态流水线

（按照同一时间内各段之间的连接方式对多功能流水线做进一步的分类）

- 静态流水线：在同一时间内，多功能流水线中的各段只能按同一种功能的连接方式工作。
  - 对于静态流水线来说，只有当输入的是一串相同的运算任务时，流水的效率才能得到充分的发挥。

例如：ASC的8段流水线

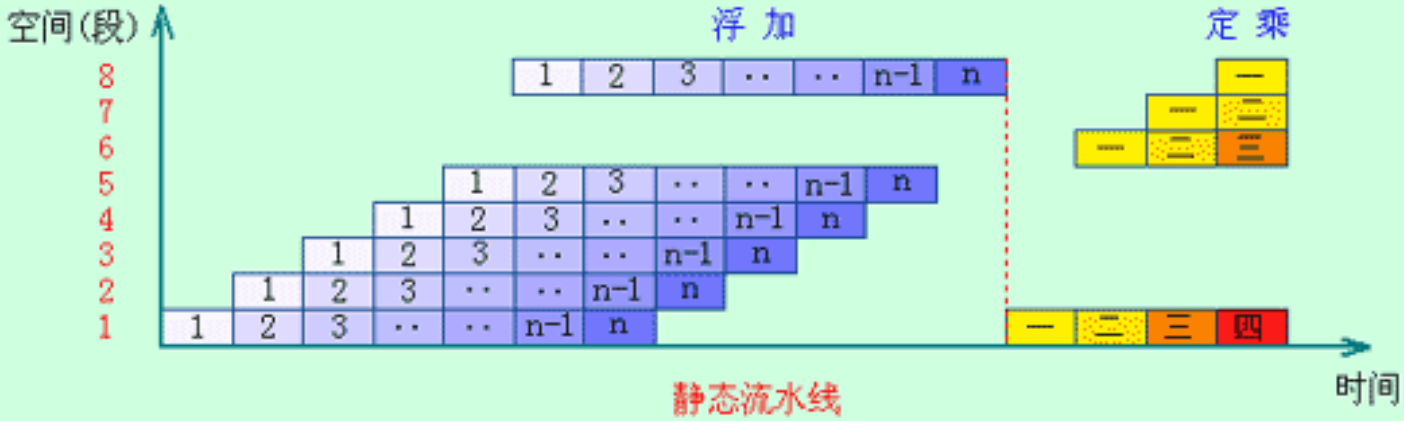
- 动态流水线：在同一时间内，多功能流水线中的各段可以按照不同的方式连接，同时执行多种功能。动画@7
  - 优点

    灵活，能够提高流水线各段的使用率，从而提高处理速度。
  - 缺点

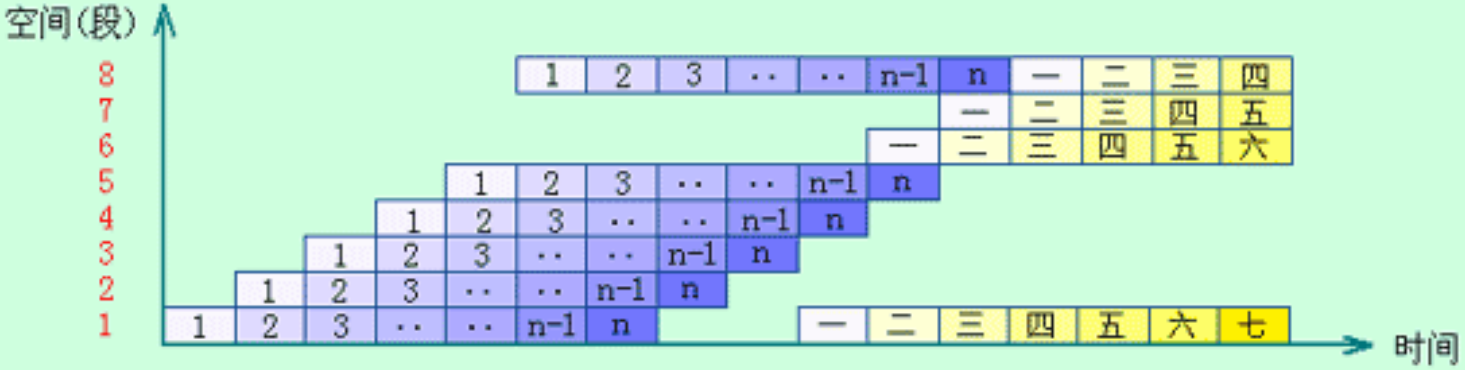
    控制复杂，目前并没被普遍采用。
- 静、动态流水线时空图的对比@8

静、动态流水线的时空图

假设该流水线要先做几个浮点加法，然后再做一批定点乘法。



静态流水线



动态流水线

3. 部件级、处理机级及处理机间流水线

（按照流水的级别来进行分类）

- 部件级流水线（运算操作流水线）：把处理机的算术逻辑运算部件分段，使得各种类型的运算操作能够按流水方式进行。

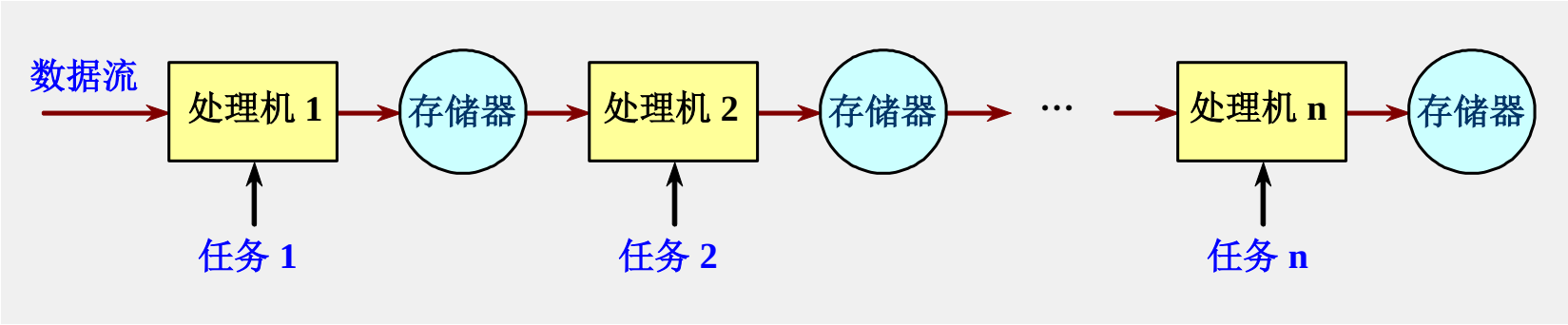
  例如：前面的浮点运算流水线

- 处理机级流水线（指令流水线）：把指令的解释执行过程按照流水方式处理。把一条指令的执行过程分解为若干个子过程，每个子过程在独立的功能部件中执行。

  例如：前面的4段指令流水线

- 处理机间流水线（宏流水线&系统级流水线）：它是由两个或两个以上的处理机串行连接起来，对同一数据流进行处理，每个处理机完成整个任务中的一部分。

动画解析@9



4. 线性流水线与非线性流水线

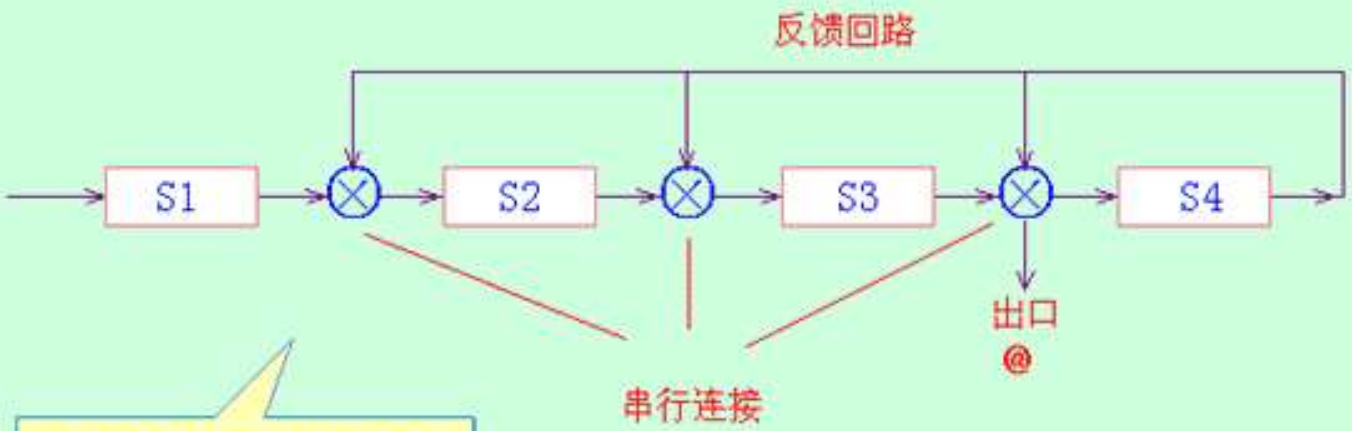
（按照流水线中是否有反馈回路来进行分类）

- 线性流水线：流水线的各段串行连接，没有反馈回路。
  - 特点：数据通过流水线中的各段时，每一个段最多只流过一次。
- 非线性流水线：流水线中除了有串行的连接外，还有反馈回路。（举例@10）
  - 非线性流水线的*调度问题*@11

    确定什么时候向流水线引进新的任务，才能使该任务不会与先前进入流水线的任务发生冲突——争用流水段。常用办法是使用预约表调度。

非线性流水线

（举 例）



虽然流水线仅由四段构成，但有些段可能要重复通过。

例如任务 @ ：

→ S1 → S2 → S3 → S4 → S2 → S3 → S4 → S3 →

5. 顺序流水线与乱序流水线

（根据任务流入和流出的顺序是否相同来进行分类）

- 顺序流水线：流水线输出端任务流出的顺序与输入端任务流入的顺序完全相同。每一个任务在流水线的各段中是一个跟着一个顺序流动的。
- 乱序流水线：流水线输出端任务流出的顺序与输入端任务流入的顺序可以不同，允许后进入流水线的任务先完成（从输出端流出）。

也称为无序流水线、错序流水线、异步流水线

6. 标量处理机与向量流水处理机

- 把指令执行部件中采用了流水线的处理机称为流水线处理机。
- 标量处理机：处理机不具有向量数据表示和向量指令，仅对标量数据进行流水处理。
- 向量流水处理机：具有向量数据表示和向量指令的处理机。

向量数据表示和流水技术的结合。

3.1 流水线的基本概念
3.2 **流水线的性能指标**
3.3 流水线的相关与冲突
3.4 流水线的实现
3.5 向量处理机

3.2 流水线的性能指标

3.2.1 吞吐率(ThroughPut)

吞吐率：在单位时间内流水线所完成的任务数量或输出结果的数量。

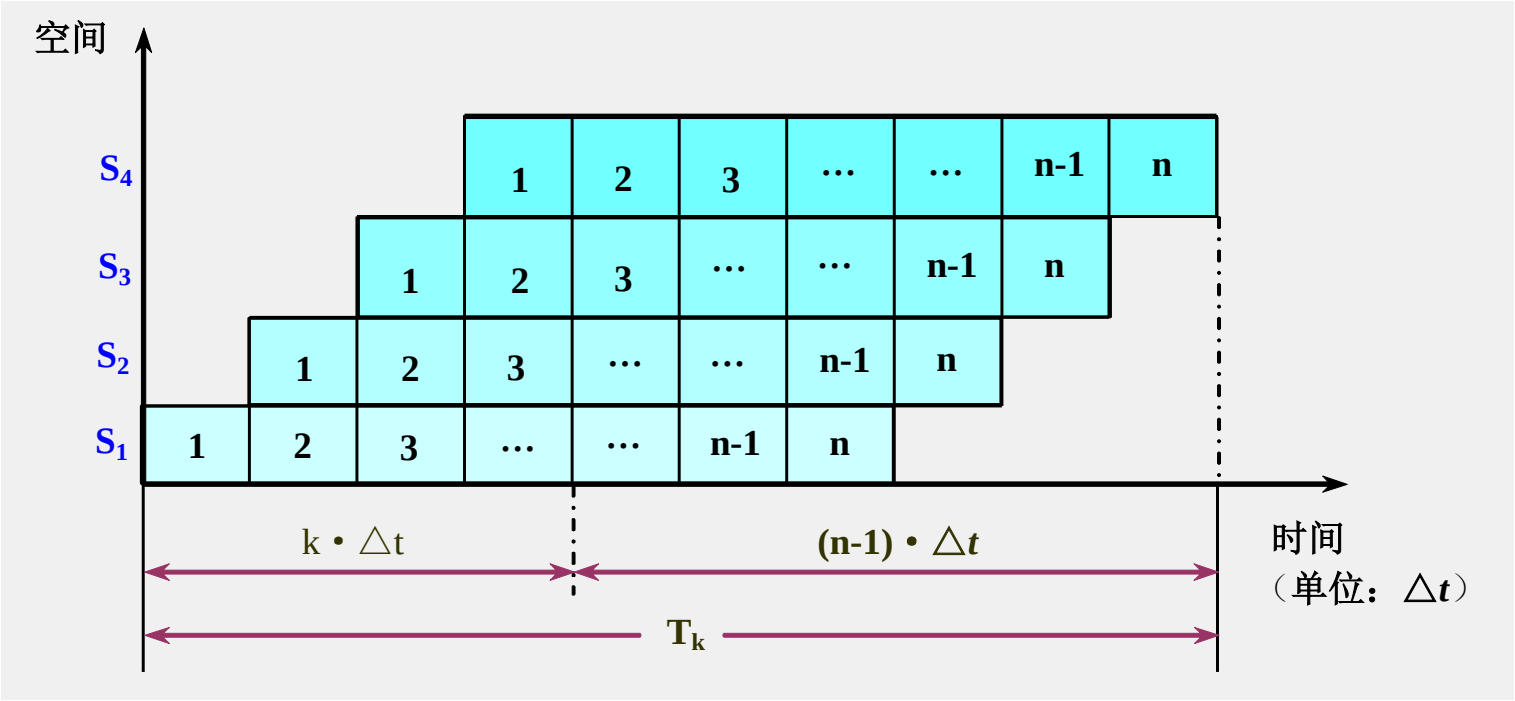
$$TP = \frac{n}{T_K}$$

n：任务数

T_k：处理完成n个任务所用的时间

1. 各段时间均相等的流水线

- 各段时间均相等的流水线时空图@12



- 流水线完成*n*个连续任务所需要的总时间@13为

  （假设一条*k*段线性流水线）

$$T_k = k\Delta t + (n-1)\Delta t = (k+n-1)\Delta t$$

- 流水线的实际吞吐率

$$TP = \frac{n}{(k+n-1)\Delta t}$$

- 最大吞吐率

$$TP_{\max} = \lim_{n\to\infty} \frac{n}{(k+n-1)\Delta t} = \frac{1}{\Delta t}$$

➢ 最大吞吐率与实际吞吐率的关系

$$TP = \frac{n}{k+n-1} TP_{\max}$$

- 流水线的实际吞吐率小于最大吞吐率，它除了与每个段的时间有关外，还与流水线的段数k以及输入到流水线中的任务数n等有关。

  问题：k越大，TP越小？怎么回事？

- 只有当$n \gg k$时，才有$TP \approx TP_{max}$。

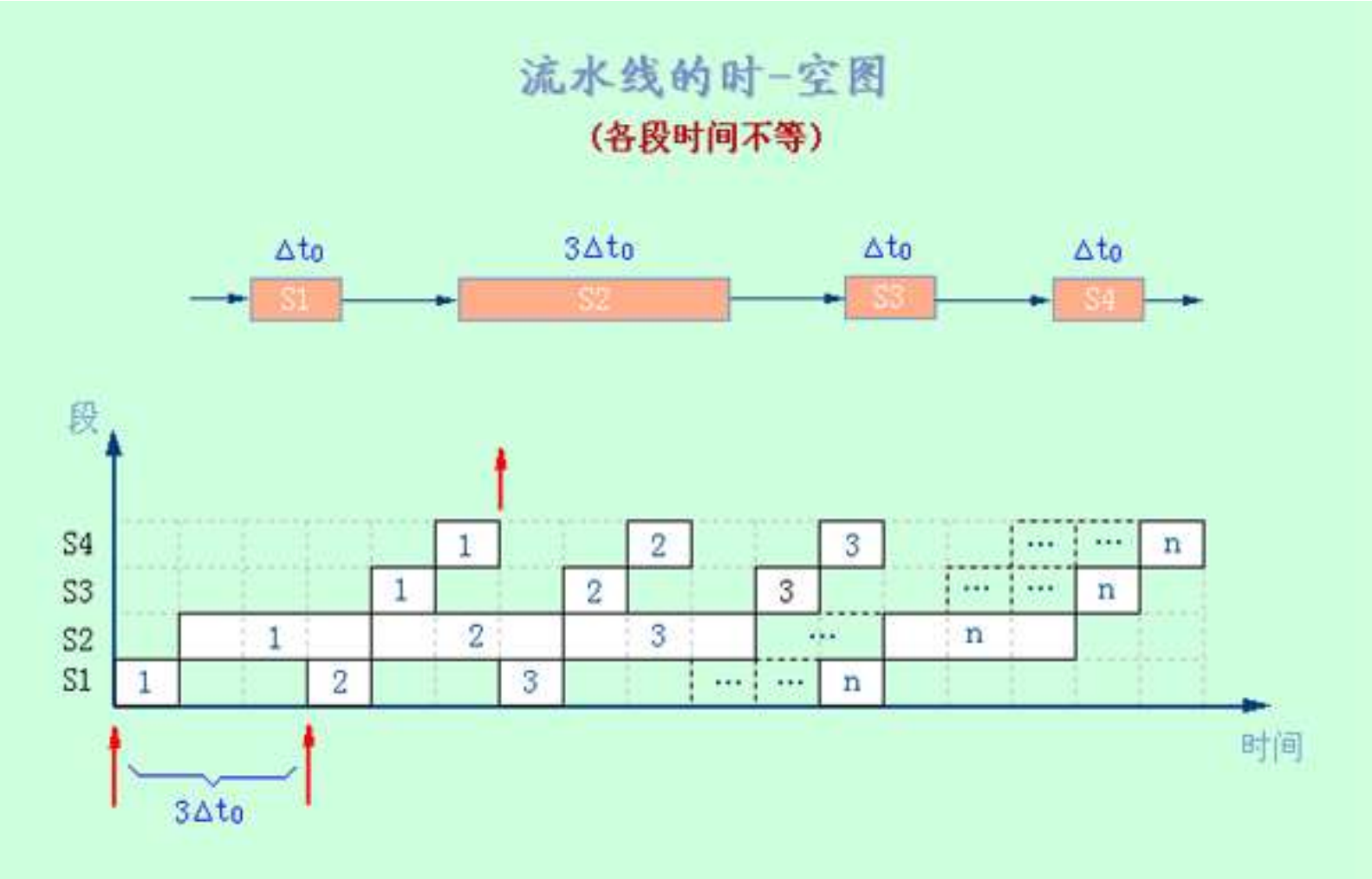
2. 各段时间不完全相等的流水线

- 各段时间不等的流水线及其时空图

  举例（时空图@14）

  - 一条4段的流水线
  - S1，S3，S4各段的时间：Δt
  - S2的时间：$3\Delta t$ （瓶颈段）

流水线中这种时间最长的段称为流水线的瓶颈段。


流水线的时-空图
(各段时间不等)
Δt0
3Δt0
Δt0
Δt0
S1
S2
S3
S4
段
时间
3Δt0

- 各段时间不等的流水线的实际吞吐率：

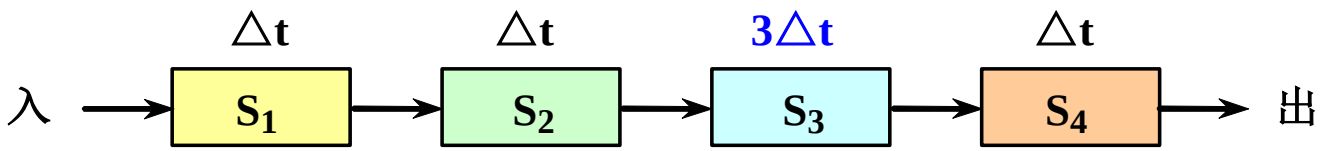
（ Δt_i为第i段的时间，共有k个段 ）

$$TP = \frac{n}{\sum_{i=1}^{k} \Delta t_i + (n-1)\max(\Delta t_1, \Delta t_2, \cdots, \Delta t_k)}$$

- 流水线的最大吞吐率为

$$TP_{\max} = \frac{1}{\max(\Delta t_1, \Delta t_2, \cdots \Delta t_k)}$$

例如：一条4段的流水线中，S_1，S_2，S_4各段的时间都是Δt，唯有S_3的时间是$3\Delta t$。



最大吞吐率为

$$TP_{\max} = \frac{1}{3\Delta t}$$

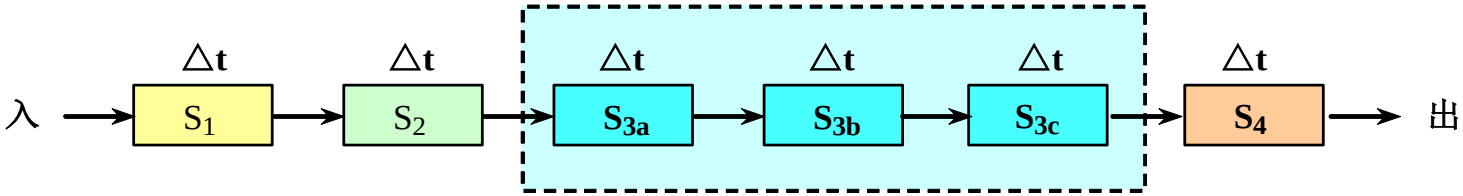
3. 解决流水线瓶颈问题的常用方法

举例@15

- 细分瓶颈段

例如：对前面的4段流水线

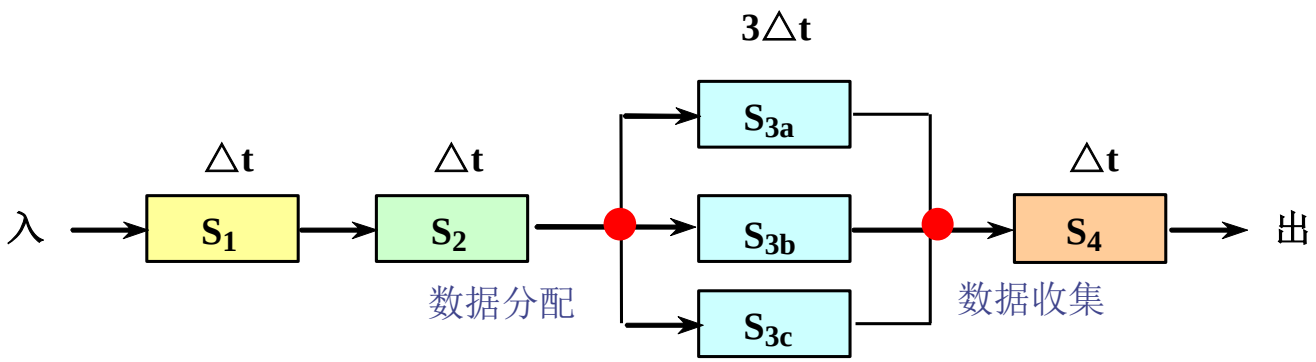
把瓶颈段S_3细分为3个子流水线段：S_{3a}，S_{3b}，S_{3c}



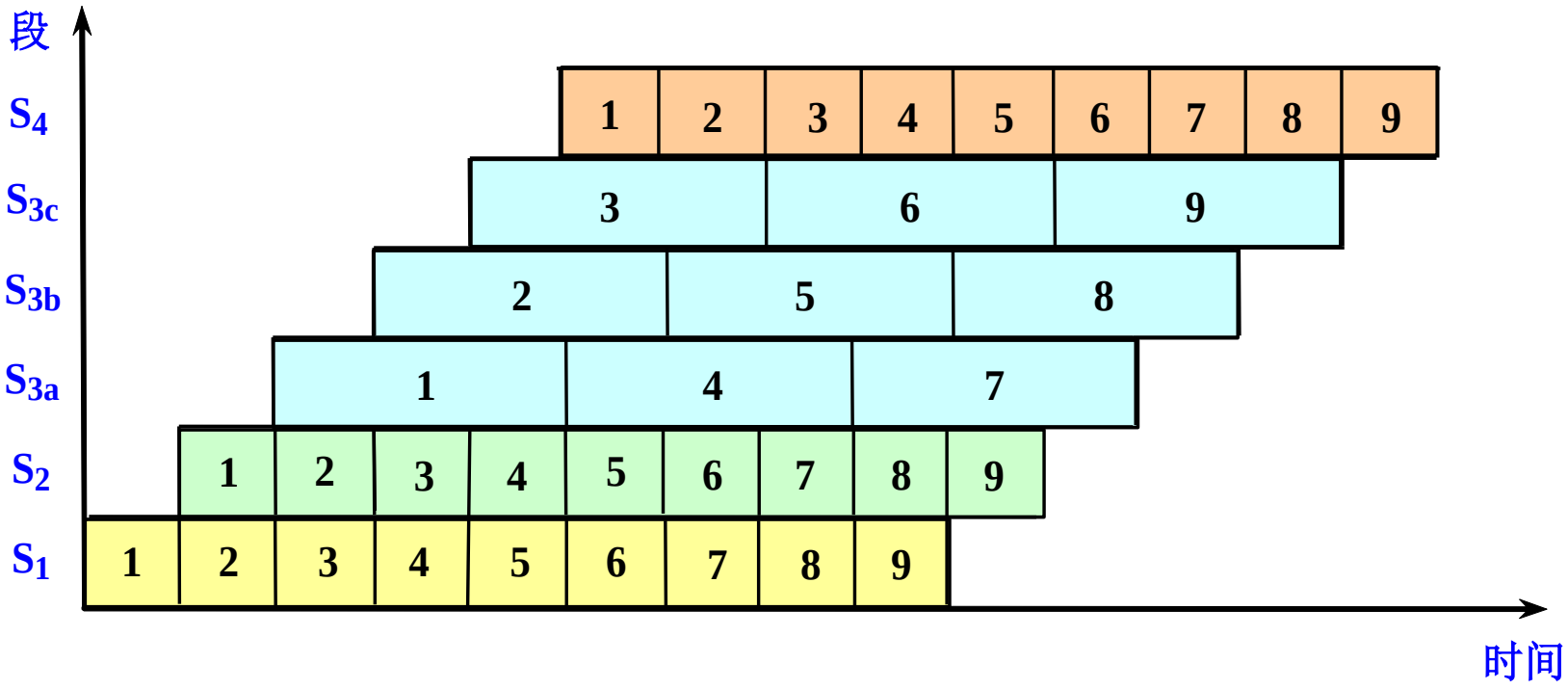
- 重复设置瓶颈段
  - 举例：时空图@16
  - 缺点：控制逻辑比较复杂，所需的硬件增加了。

例如：对前面的4段流水线

重复设置瓶颈段S_3：S_{3a}，S_{3b}，S_{3c}



改进后的流水线的吞吐率：$TP_{max} = \frac{1}{\Delta t}$



重复设置瓶颈段后的时空图

3.2.2 加速比(Speedup)

加速比：完成同样一批任务，不使用流水线所用的时间与使用流水线所用的时间之比。

假设：不使用流水线所用的间为T_s，使用流水线后所用的时间为T_k，则该流水线的加速比为

$$S = \frac{T_s}{T_k}$$

1. 流水线各段时间相等（都是△ t）

- 一条k段流水线完成n个连续任务

  所需要的时间为

$$T_k = (k+n-1)\Delta t$$

- 顺序执行n个任务

  所需要的时间： $T_s = nk\Delta t$ （解释@17）

- 流水线的实际加速比为

$$S = \frac{nk}{k+n-1}$$

- 最大加速比

$$S_{\max} = \lim_{n\to\infty} \frac{nk}{k+n-1} = k$$

当n>>k时，$S \approx k$

思考：流水线的段数愈多愈好？

2. 流水线的各段时间不完全相等时

- 一条k段流水线完成n个连续任务的实际加速比为

$$S=\frac{n\sum_{i=1}^{k}\Delta t_i}{\sum_{i=1}^{k}\Delta t_i+(n-1)\max(\Delta t_1,\Delta t_2,\cdots,\Delta t_k)}$$

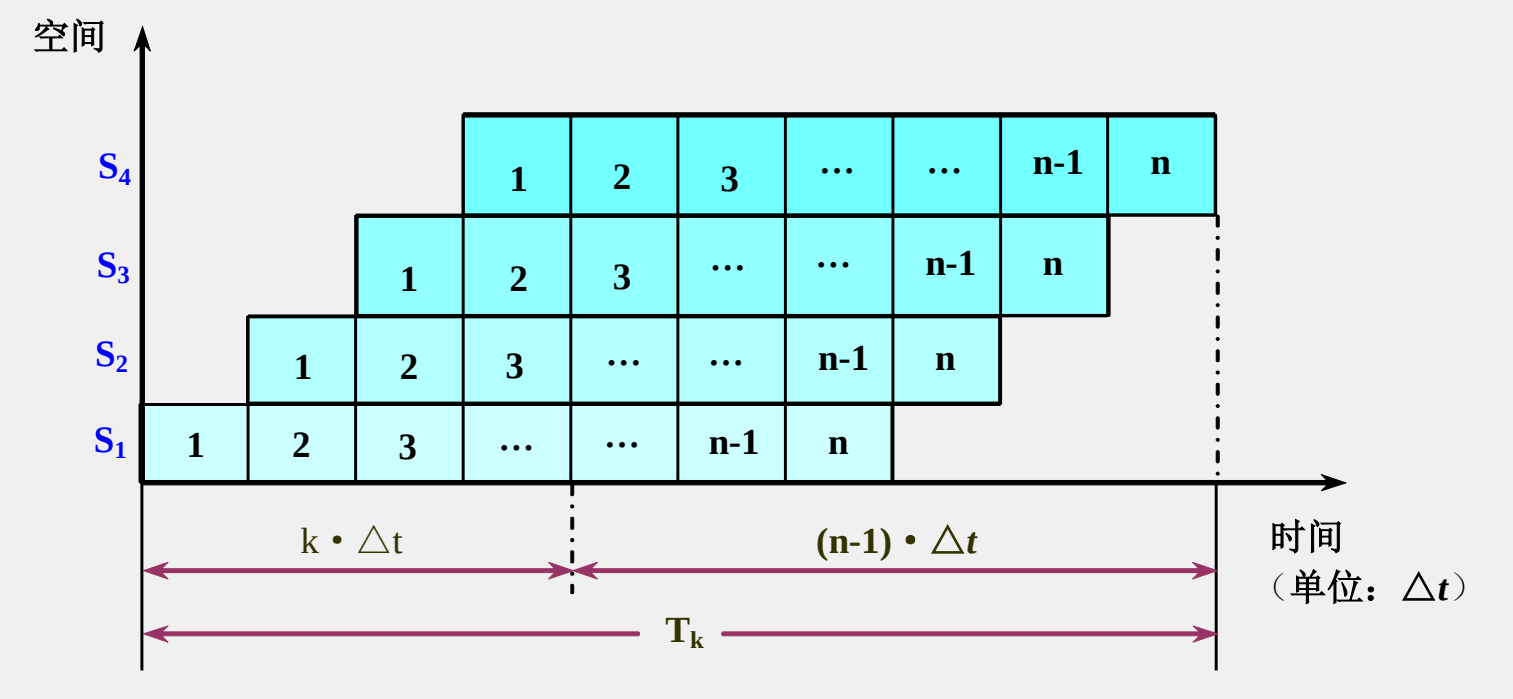
3.2.3 效率(Efficiency)

效率：流水线中的设备实际使用时间与整个运行时间的比值，即流水线**设备的利用率**。

由于流水线有通过时间和排空时间，所以在连续完成n个任务的时间内，各段并不是满负荷地工作。

1. 各段时间相等

- 各段的效率e_i相同



$$e_1 = e_2 = \cdots = e_k = \frac{n\Delta t}{T_k} = \frac{n}{k+n-1}$$

- 整条流水线的效率为

$$E=\frac{e_1+e_2+\cdots+e_k}{k}=\frac{ke_1}{k}=\frac{kn\Delta t}{kT_k}$$

- 可以写成

$$E=\frac{n}{k+n-1}$$

- 最高效率为

$$E_{\max}=\lim_{n\to\infty}\frac{n}{k+n-1}=1$$

当n>>k时，$E\approx1$。

- 当流水线各段时间相等时，**流水线的效率*E*与吞吐率*TP*成正比。**

$$E = TP\triangle t$$

2. 流水线的效率是流水线的实际加速比*S*与它的最大加速比*k*的比值。

$$E = \frac{S}{k}$$

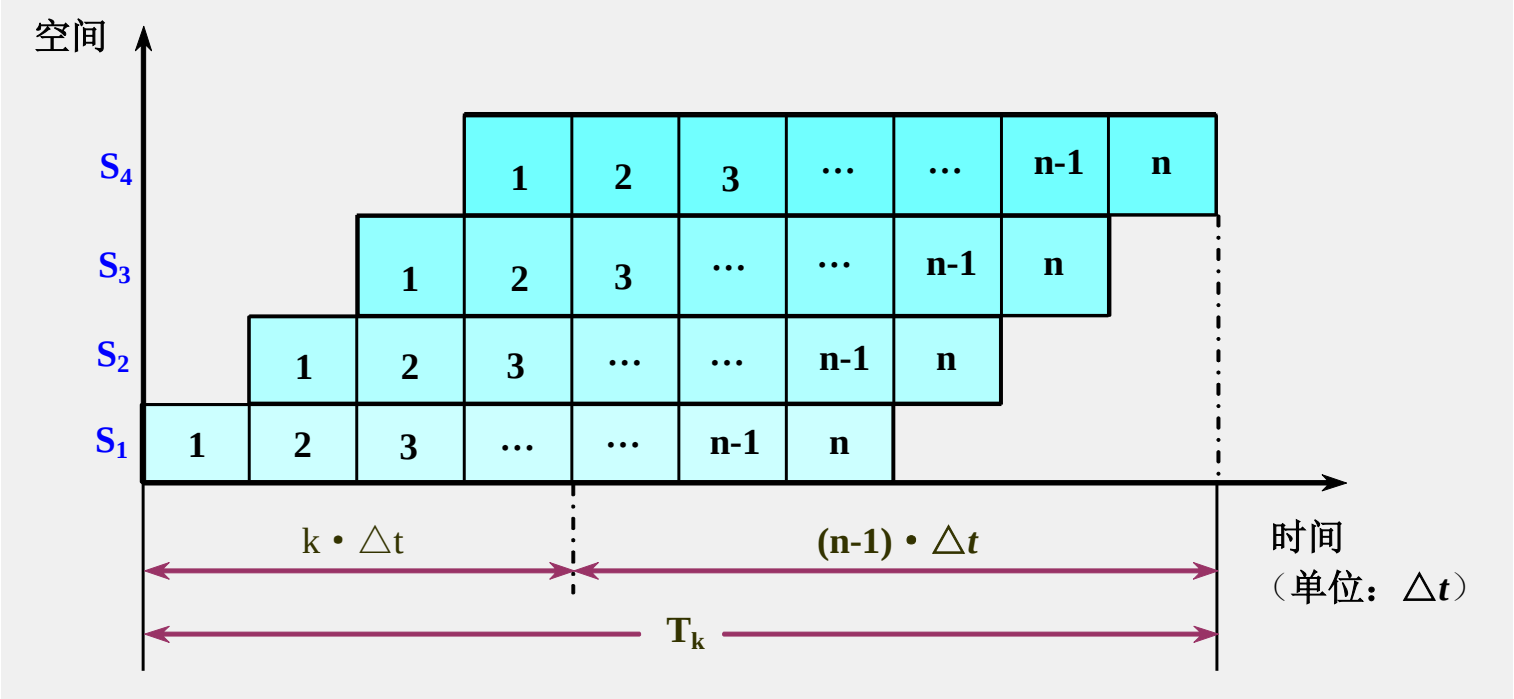
当*E*=1时，*S*=*k*，实际加速比达到最大。

3. 从时空图上看，效率就是n个任务占用的时空面积和k个段总的时空面积之比。

$$E=\frac{n\text{个任务实际占用的时空区}}{k\text{个段总的时空区}}$$

当各段时间不相等时 举例@18

$$E=\frac{n\cdot\sum_{i=1}^{k}\Delta t_i}{k\left[\sum_{i=1}^{k}\Delta t_i+(n-1)\cdot\max(\Delta t_1,\Delta t_2,\cdots,\Delta t_k)\right]}$$

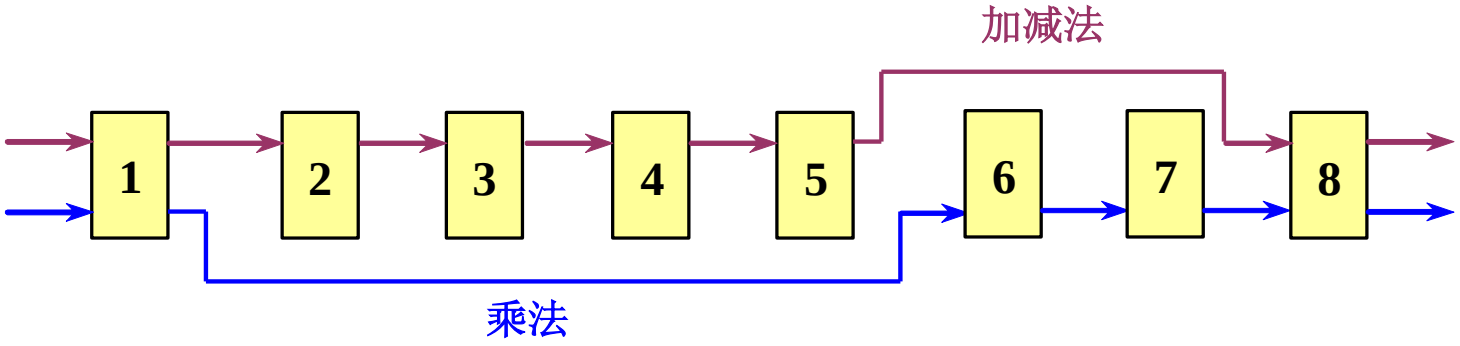

空间
S4
S3
S2
S1
1 2 3 … … n-1 n
k • △t
(n-1) • △t
Tk
时间
（单位：△t）

3.2.4 流水线的性能分析举例

例3.1 设在下图所示的静态流水线上计算：

$$\prod_{i=1}^{4}(A_i + B_i)$$

流水线的输出可以直接返回输入端或暂存于相应的流水寄存器中，试计算其吞吐率、加速比和效率。

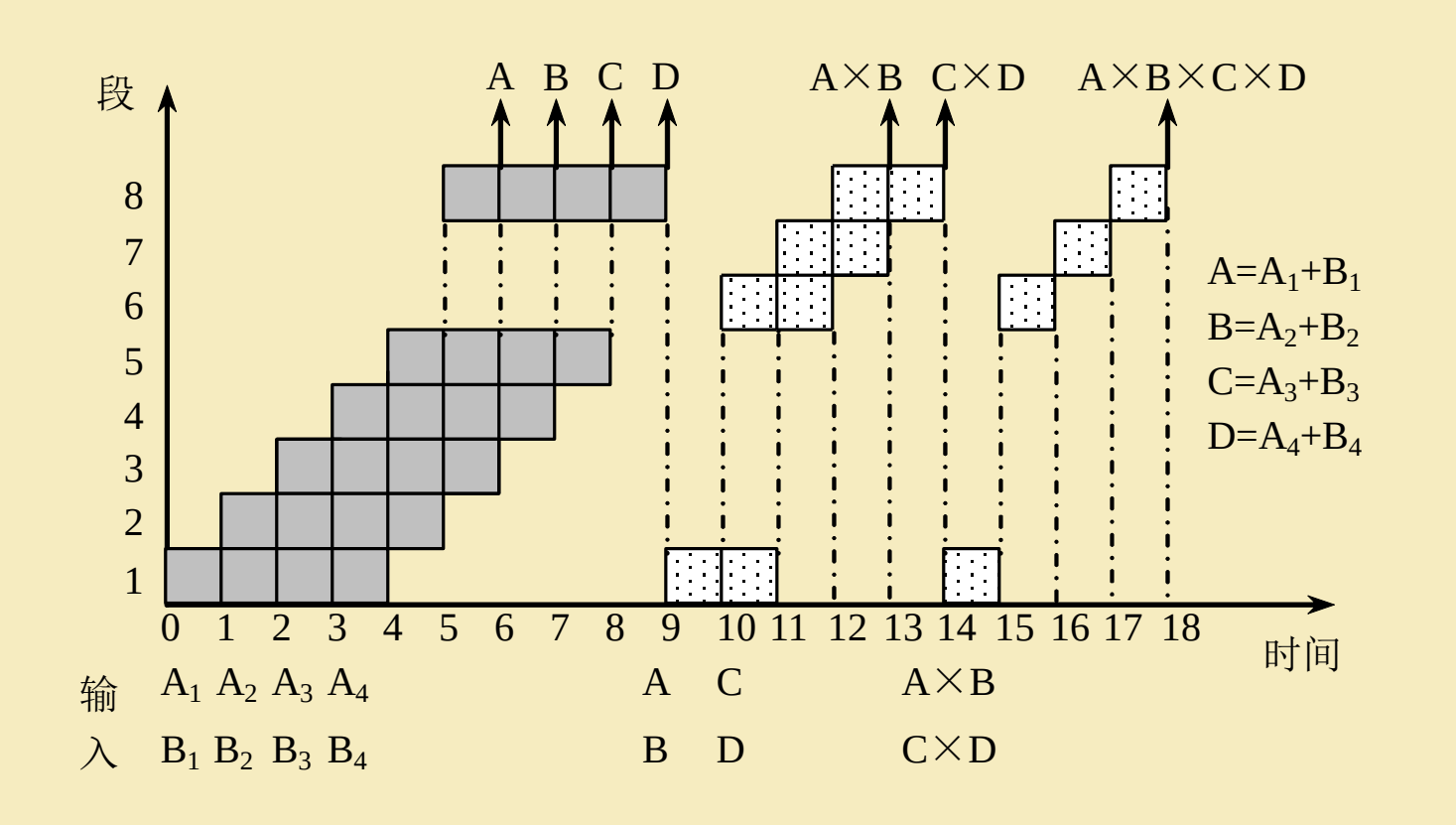


（每段的时间都为△ t）

解：（1）选择适合于流水线工作的算法

- 先计算A_1+B_1、A_2+B_2、A_3+B_3和A_4+B_4；--4次加法
- 再计算$(A_1+B_1)\times(A_2+B_2)$和$(A_3+B_3)\times(A_4+B_4)$；--2次乘法
- 然后求总的乘积结果。--1次乘法

（2）画出时空图



$$TP = \frac{7}{18\Delta t} \qquad S = \frac{36\Delta t}{18\Delta t} = 2 \qquad E = \frac{4\times 6 + 3\times 4}{8\times 18} = 0.25$$

（3）计算性能

- 在18个△t时间中，给出了7个结果。吞吐率为：

$$TP = \frac{7}{18\Delta t}$$

- 不用流水线，由于一次求和需6△t，一次求积需4△t，则产生上述7个结果共需（4×6+3×4）△t = 36△t

加速比为

$$S = \frac{36\Delta t}{18\Delta t} = 2$$

- 流水线的效率

$$E=\frac{4\times 6+3\times 4}{8\times 18}=0.25$$

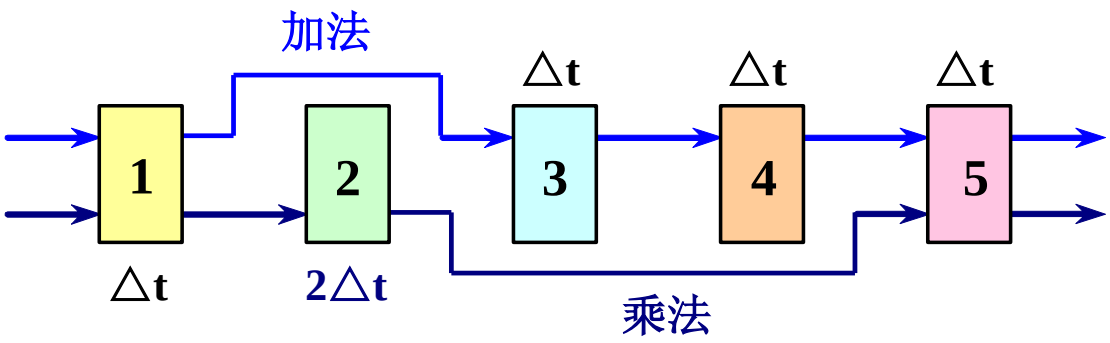
可以看出，在求解此问题时，该流水线的效率不高。

- 主要原因
  - 多功能流水线在做某一种运算时，总有一些段是空闲的。
  - 静态流水线在进行功能切换时，要等前一种运算全部流出流水线后才能进行后面的运算。
  - 运算之间存在关联，后面有些运算要用到前面运算的结果。
  - 流水线的工作过程有建立与排空部分。

例3.2 有一条动态多功能流水线由5段组成，加法用1、3、4、5段，乘法用1、2、5段，第2段的时间为2△t，其余各段时间均为△t，而且流水线的输出可以直接返回输入端或暂存于相应的流水寄存器中。若在该流水线上计算：

$$\sum_{i=1}^{4}(A_i \times B_i)$$

试计算其吞吐率、加速比和效率。

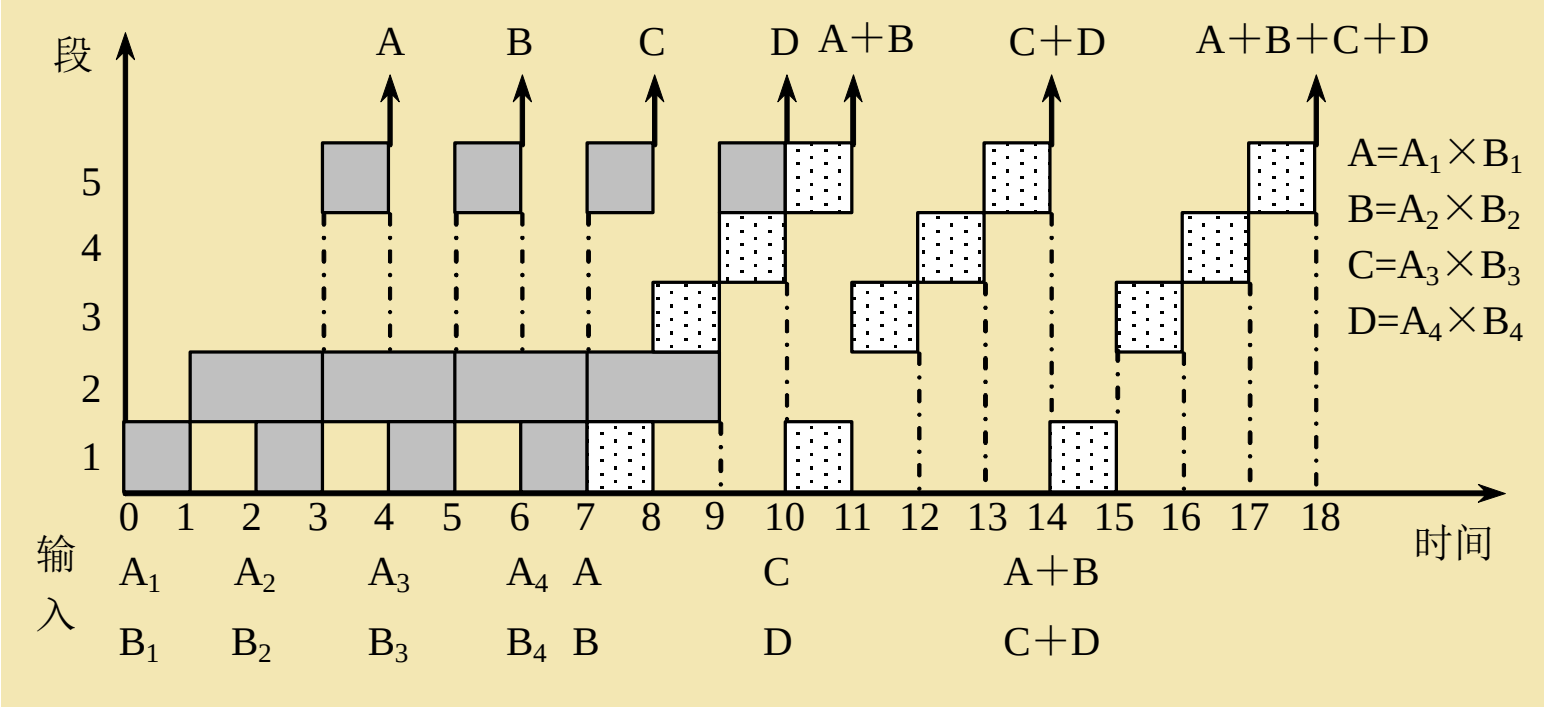


解：(1) 选择适合于流水线工作的算法

- 应先计算$A1\times B1$、$A2\times B2$、$A3\times B3$和$A4\times B4$；--4次乘法
- 再计算$(A1\times B1)+(A2\times B2)$
  $(A3\times B3)+(A4\times B4)$；--2次加法
- 然后求总的累加结果。--1次加法

(2) 画出时空图

(3) 计算性能



$$TP = \frac{7}{18\Delta t} \qquad S = \frac{28\Delta t}{18\Delta t} \approx 1.56 \qquad E = \frac{4\times 4 + 3\times 4}{5\times 18} \approx 0.31$$

3.2.5 流水线设计中的若干问题

1. 瓶颈问题
   - 理想情况下，流水线在工作时，其中的任务是同步地每一个时钟周期往前流动一段。
   - 当流水线各段不均匀时，机器的时钟周期取决于瓶颈段的延迟时间。
   - 在设计流水线时，要尽可能使各段时间相等。
2. 流水线的额外开销
   - 流水寄存器延迟
   - 时钟偏移开销

- 流水寄存器需要建立时间和传输延迟
  - 建立时间：在触发写操作的时钟信号到达之前，寄存器输入必须保持稳定的时间。
  - 传输延迟：时钟信号到达后到寄存器输出可用的时间。
- 时钟偏移开销
  - 流水线中，时钟到达各流水寄存器的最大差值时间。（时钟到达各流水寄存器的时间不是完全相同）

- 几个问题
  - 流水线并不能减少（而且一般是增加）单条指令的执行时间，但却能提高吞吐率。
  - 增加流水线的深度（段数）可以提高流水线的性能。
  - 流水线的深度受限于流水线的额外开销。
  - 当时钟周期小到与额外开销相近时，流水已没意义。因为这时在每一个时钟周期中已没有时间来做有用的工作。

3. 冲突问题

指令流水线中的指令如存在关联，就可能需要流水线的停顿(stall)以等待前面指令的结果，这是流水线设计中要解决的重要问题之一。

3.1 流水线的基本概念
3.2 流水线的性能指标
3.3 **流水线的相关与冲突**
3.4 流水线的实现
3.5 向量处理机

3.3 流水线的相关与冲突

3.3.1 一个经典的5段流水线

- 介绍一个经典的5段RISC流水线
- 首先讨论在非流水情况下是如何实现的

1. 一条指令的执行过程分为以下5个周期：

- 取指令周期（IF）
  - IR ← Mem[PC] 。
  - PC值加4。（假设每条指令占4个字节）

- 指令译码/读寄存器周期（ID）
  - 译码。
  - 用IR中的寄存器编号去访问通用寄存器组，读出所需的操作数。
- 执行/有效地址计算周期（EX）

不同指令所进行的操作不同：

- 存储器访问指令：ALU把所指定的寄存器的内容与偏移量相加，形成用于访存的有效地址。
- 寄存器－寄存器ALU指令：ALU按照操作码指定的操作对从通用寄存器组中读取的数据进行运算。

- 寄存器－立即数ALU指令：ALU按照操作码指定的操作对从通用寄存器组中读取的第一操作数和立即数进行运算。
- 分支指令：ALU把偏移量与PC值相加，形成转移目标的地址。同时，对在前一个周期读出的操作数进行判断，确定分支是否成功。

➢ 存储器访问／分支完成周期（MEM）

该周期处理的指令只有load、store和分支指令。其他类型的指令在此周期不做任何操作。

- load和store指令

load指令：用上一个周期计算出的有效地址从存储器中读出相应的数据。

store指令：把指定的数据写入这个有效地址所指出的存储器单元。

- 分支指令

分支“成功”，就把转移目标地址送入PC。

分支指令执行完成。

- 写回周期（WB）

  ALU运算指令和load指令在这个周期把结果数据写入通用寄存器组。

  ALU运算指令：结果数据来自ALU。

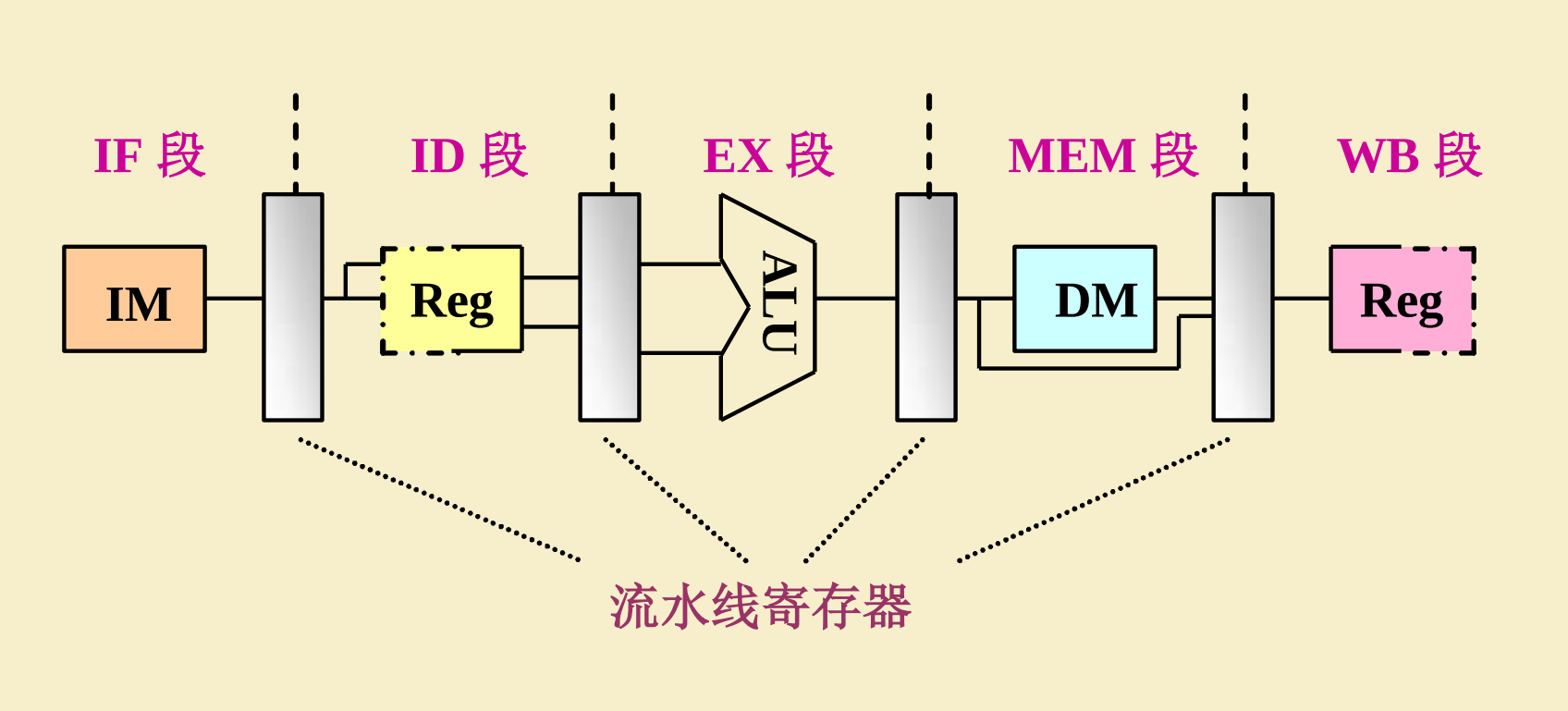
  load指令：结果数据来自存储器系统。

在这个实现方案中：

- 分支指令需要4个时钟周期（如果把分支指令的执行提前到ID周期，则只需要2个周期）。
- store指令需要4个周期。
- 其他指令需要5个周期才能完成。

2. 将上述实现方案修改为流水线实现

- 一个经典的5段流水线
  - 每一个周期作为一个流水段。
  - 在各段之间加上锁存器（流水寄存器）。

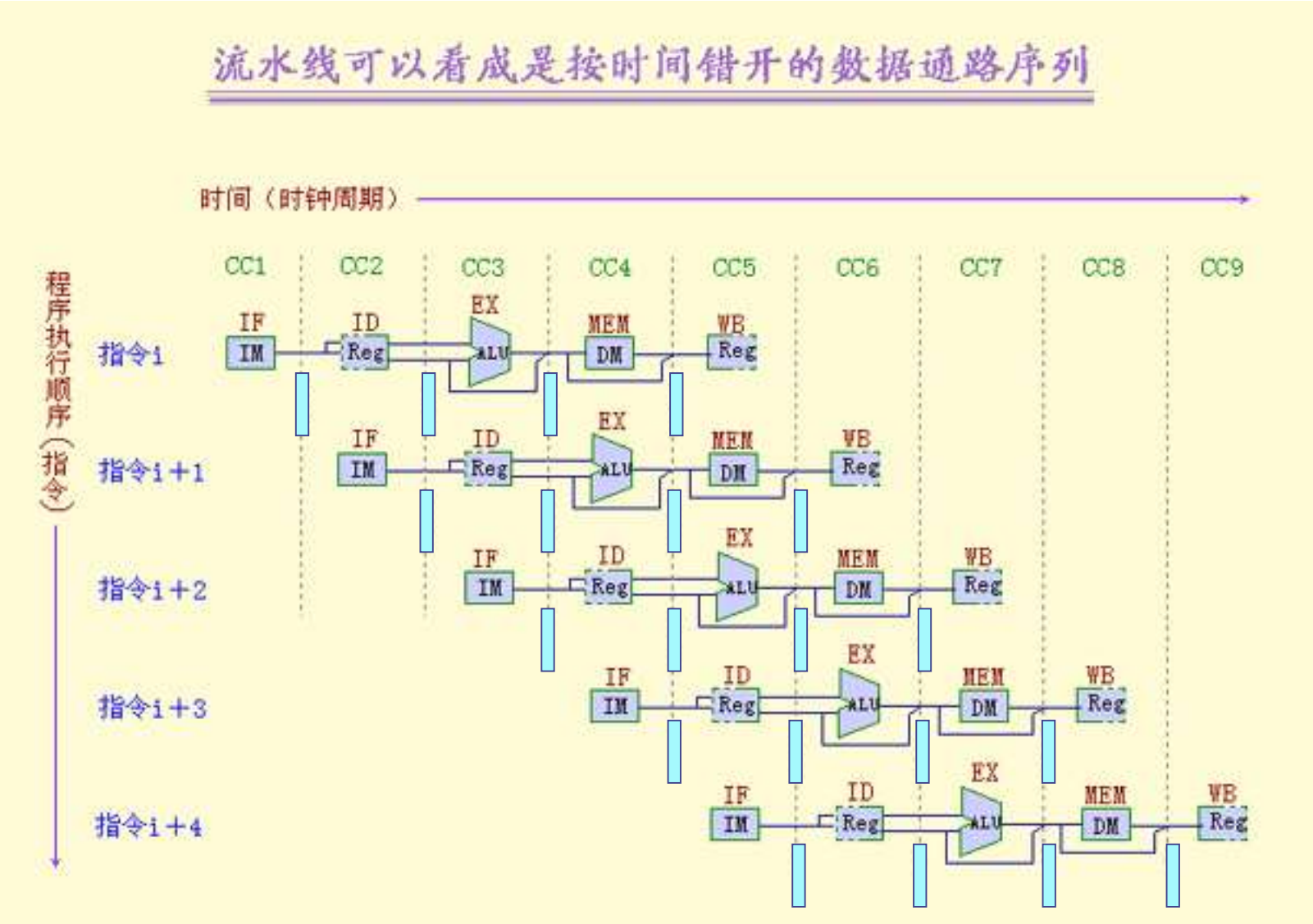


- 5段流水线的两种描述方式
  - 第一种描述（类似于时空图）

	时钟周期								
指令编号	1	2	3	4	5	6	7	8	9
指令i	IF	ID	EX	MEM	WB				
指令i+1		IF	ID	EX	MEM	WB			
指令i+2			IF	ID	EX	MEM	WB		
指令i+3				IF	ID	EX	MEM	WB	
指令i+4					IF	ID	EX	MEM	WB

动画@19

➢ 第二种描述（按时间错开的数据通路序列）



动画@20

3. 采用流水线方式实现时，应解决以下几个问题：

- 要保证不会在同一时钟周期要求同一个功能段做两件不同的工作。

  例如，不能要求ALU同时做有效地址计算和算术运算。

- 避免IF段的访存（取指令）与MEM段的访存（读/写数据）发生冲突。
  - 可以采用分离的指令存储器和数据存储器；
  - 一般采用分离的指令Cache和数据Cache。
- 同样的时钟周期，改为流水执行后，存储器带宽要提升到原来的5倍才能满足流水取值的要求。

3. 采用流水线方式实现时，应解决以下几个问题：(续)

➢ ID段和WB段可能访问同一寄存器。

ID段：读通用寄存器组　　WB段：写通用寄存器组

如何解决对同一寄存器的访问冲突？

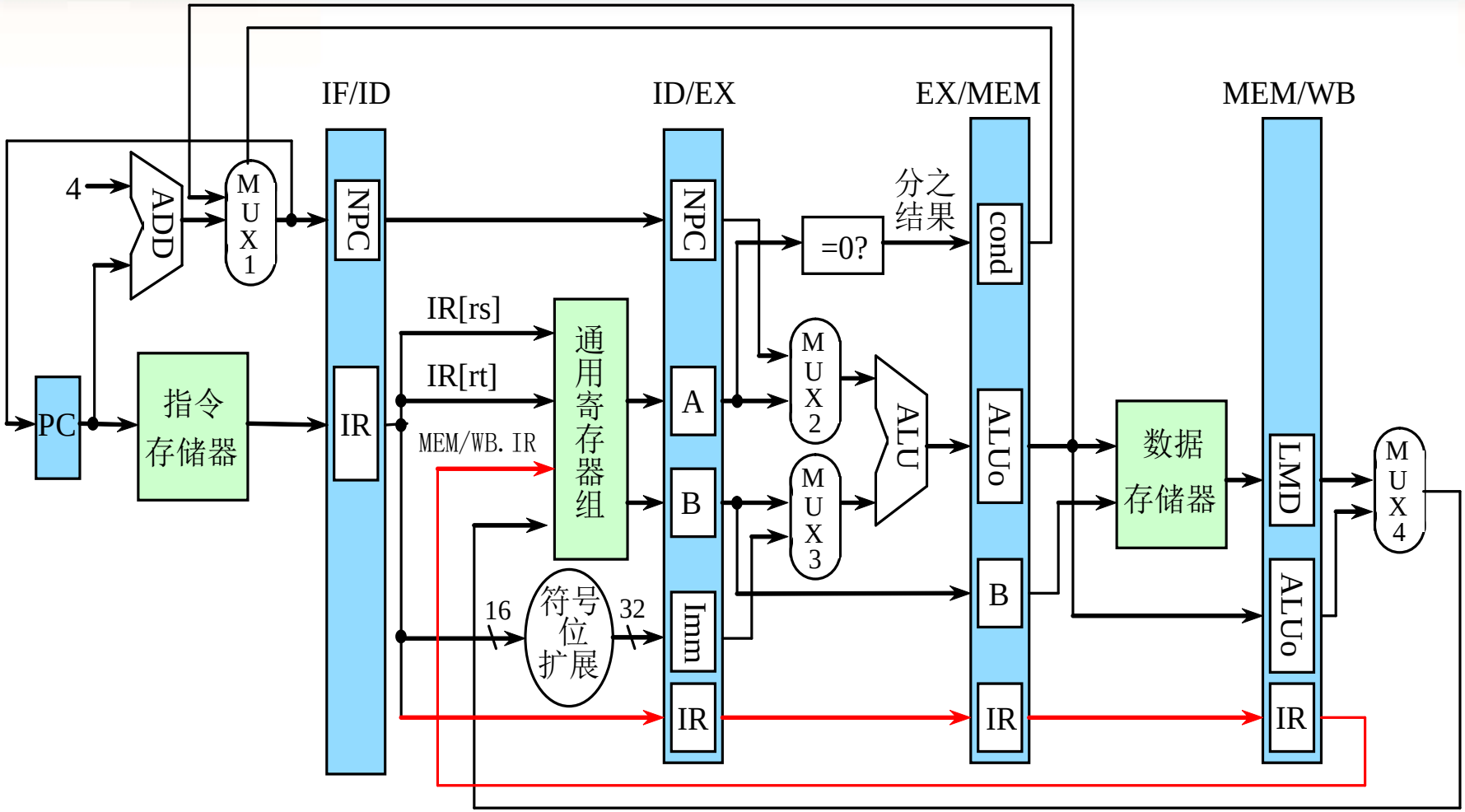
把写操作安排在时钟周期的前半拍完成，把读操作安排在后半拍完成。

3. 采用流水线方式实现时，应解决以下几个问题：(续)

- 考虑PC的问题
  - 流水线为了能够每个时钟周期启动一条新的指令，就必须在每个时钟周期进行PC值的加4操作，并保留新的PC值。这种操作必须在IF段完成，以便为取下一条指令做好准备。

    （需设置一个专门的加法器）
  - 但分支指令也可能改变PC的值，而且是在MEM段进行，这会导致冲突。

请考虑一下，如何处理分支指令？


IF/ID
ID/EX
EX/MEM
MEM/WB
4
ADD
MUX1
NPC
IR
PC
指令存储器
IR[rs]
IR[rt]
MEM/WB. IR
通用寄存器组
16
符号位扩展
32
NPC
A
B
Imm
IR
=0?
分之结果
MUX2
MUX3
ALU
cond
ALUo
B
IR
数据存储器
LMD
ALUo
IR
MUX4

3.3.2 相关与流水线冲突

1. 相关

- 相关：两条指令之间存在某种依赖关系。

  如果两条指令相关，则它们就有可能不能在流水线中重叠执行或者只能部分重叠执行。

- 相关有3种类型
  - 数据相关（也称真数据相关）
  - 名相关
  - 控制相关

（1）数据相关

- 对于两条指令*i*（在前，下同）和*j*（在后，下同），如果下述条件之一成立，则称指令*j*与指令*i*数据相关。
  - 指令*j*使用指令*i*产生的结果；
  - 指令*j*与指令*k*数据相关，而指令*k*又与指令*i*数据相关。
- 数据相关具有传递性。

数据相关反映了数据的<u>流动关系</u>，即如何从其产生者流动到其消费者。

例如：下面这一段代码存在数据相关。

```
Loop:  L.D     F0, 0(R1)      // F0为数组元素
       ADD.D   F4, F0, F2     // 加上F2中的值
       S.D     F4, 0(R1)      // 保存结果
       DADDIU  R1, R1, -8     // 数组指针递减8个字节
       BNE     R1, R2, Loop   // 如果R1≠R2，则分支
```

- 当数据的流动是经过寄存器时，相关的检测比较直观和容易。
- 当数据的流动是经过存储器时，检测比较复杂。
  - 相同形式的地址其有效地址未必相同。

    例如：形式都是**0(R1)**，但不同的**R1**值对应不同的有效地址。
  - 形式不同的地址其有效地址却可能相同。

（2）名相关

- 名：指令所访问的寄存器或存储器单元的名称。
- 如果两条指令使用相同的名，但是它们之间并没有数据流动，则称这两条指令存在名相关。

（2）名相关 （续）

- 指令*j*与指令*i*之间的名相关有两种：
  - **反相关：**如果指令*j*写的名与指令*i*读的名相同，则称指令*i*和*j*发生了反相关。-- 读后写

    指令*j*写的名＝指令*i*读的名
  - **输出相关：**如果指令*j*和指令*i*写相同的名，则称指令*i*和*j*发生了输出相关。要保证最后的数据是*j*写的，执行顺序非常重要。 -- 写后写

    指令*j*写的名＝指令*i*写的名

（2）名相关 （续）

- 名相关的两条指令之间并没有数据的传送。
- 指令的执行顺序很重要
- 如果一条指令中的名改变了，并不影响另外一条指令的执行。
- 换名技术
  - 换名技术：通过改变指令中操作数的名来消除名相关。
  - 对于寄存器操作数进行换名称为寄存器换名。

既可以用编译器静态实现，也可以用硬件动态完成。

例如：考虑下述代码：

```
DIV.D    F2，F6，F4   ；读F6
ADD.D    F6，F0，F12 ；写F6
SUB.D    F8，F6，F14
```

DIV.D和ADD.D存在反相关。

进行寄存器换名（F6换成S）后，变成：

```
DIV.D    F2，F6，F4
ADD.D    S，F0，F12
SUB.D    F8，S，F14
```

（3）控制相关

- 控制相关是指由分支指令引起的相关。
  - 为了保证程序应有的执行顺序，必须严格按控制相关确定的顺序执行。
- 典型的程序结构是“if-then”结构。
- 请看一个示例：

S1与p1控制相关；

S2与p2控制相关；

S与p1、p2均无关。

```
if p1 {
        S1;
    };
S;
if p2 {
        S2;
    };
```

- 控制相关带来了以下两个限制：
  - 与一条分支指令控制相关的指令不能被移到该分支之前，否则这些指令就不受该分支控制了。

    对于上述的例子，then 部分中的指令不能移到if语句之前。
  - 如果一条指令与某分支指令不存在控制相关，就不能把该指令移到该分支之后。

  对于上述的例子，不能把S移到if语句的then部分中。

2. 流水线冲突

流水线冲突是指对于具体的流水线来说，由于相关的存在，使得指令流中的下一条指令不能在指定的时钟周期执行。

流水线冲突有3种类型：

- **结构冲突：**因硬件资源满足不了指令重叠执行的要求而发生的冲突。
- **数据冲突：**当指令在流水线中重叠执行时，因需要用到前面指令的执行结果而发生的冲突。
- **控制冲突：**流水线遇到分支指令和其他会改变PC值的指令所引起的冲突。

带来的几个问题：

- 导致错误的执行结果。
- 流水线可能会出现停顿，从而降低流水线的效率和实际的加速比。
- 我们约定

  当一条指令被暂停时，在该暂停指令之后流出的所有指令都要被暂停，而在该暂停指令之前流出的指令则继续进行（否则就永远无法消除冲突）。

（1）结构冲突

- 在流水线处理机中，为了能够使各种组合的指令都能顺利地重叠执行，需要对功能部件进行流水或重复设置资源。
- 如果某种指令组合因为资源冲突而不能正常执行，则称该处理机有结构冲突。
- 常见的导致结构冲突的原因：
  - **功能部件不是完全流水**
  - **资源份数不够**

- 结构冲突举例：访存冲突

有些流水线处理机只有一个存储器，将数据和指令放在一起，访存指令会导致访存冲突@21。

- 解决办法Ⅰ：插入暂停周期@22

（“流水线气泡”或“气泡”）

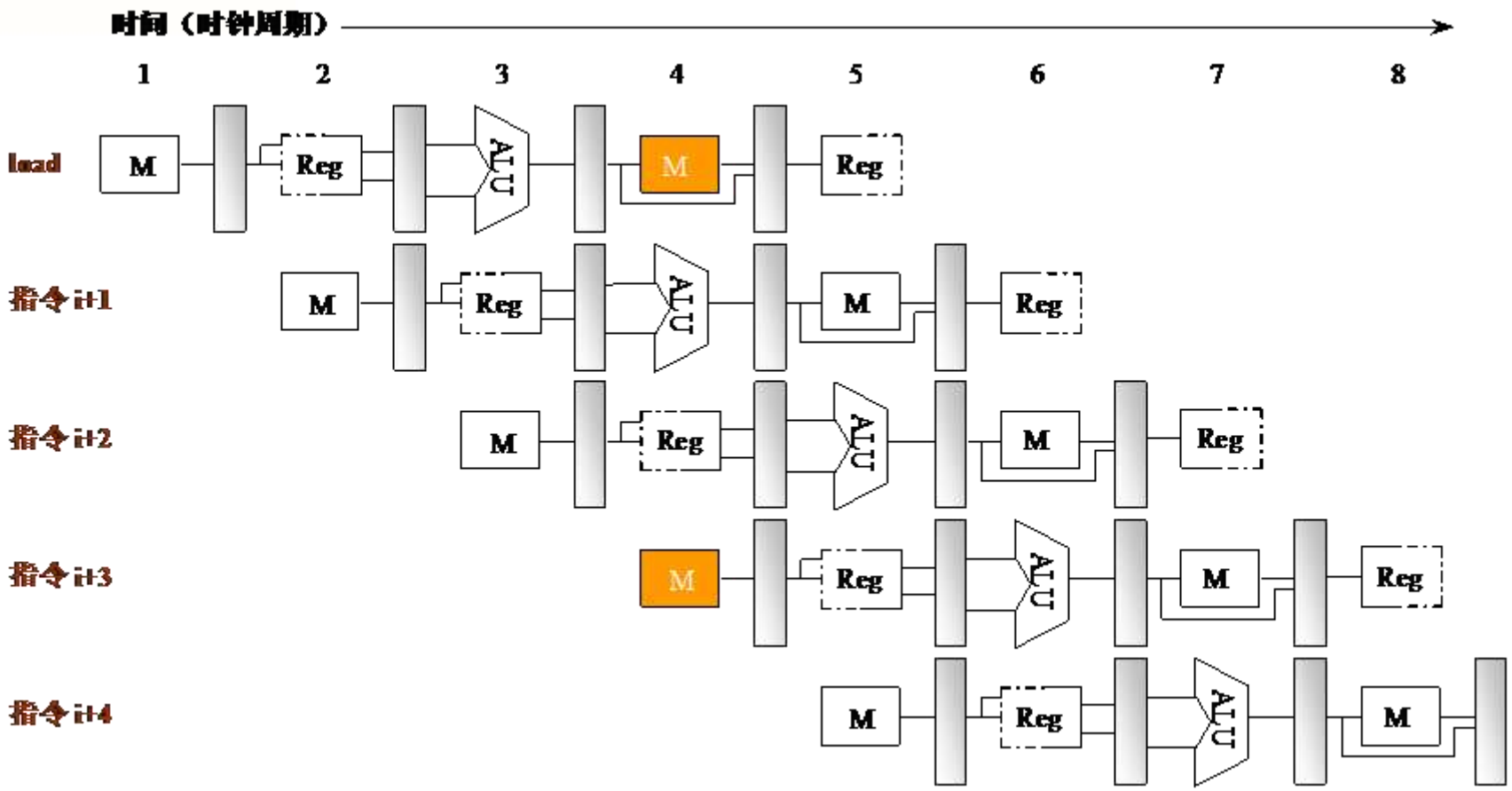
引入暂停后的时空图@23

缺点：影响流水线性能

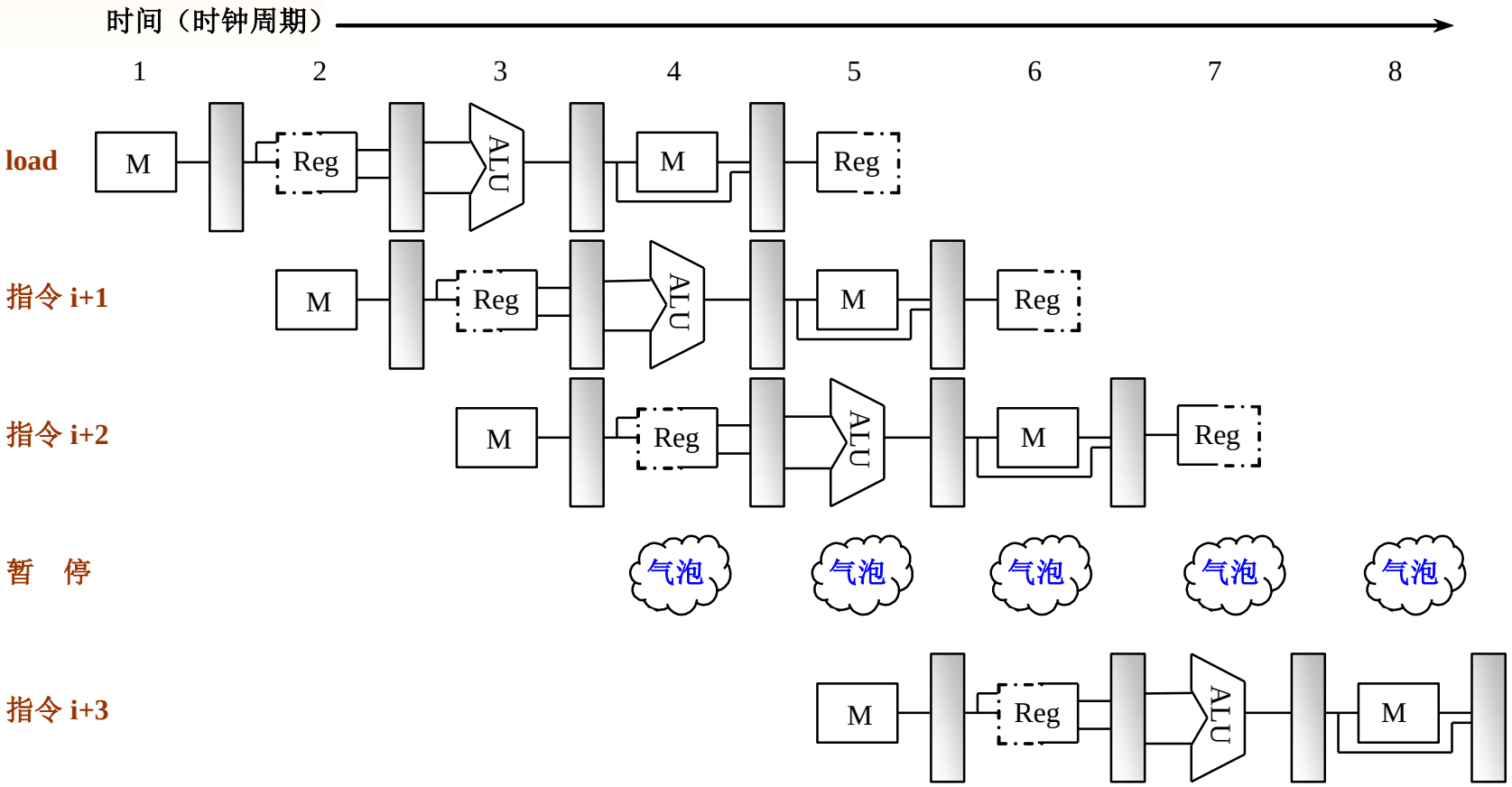
- 解决方法Ⅱ：

设置相互独立的指令存储器和数据存储器

或设置相互独立的指令Cache和数据Cache。



由于访问同一个存储器而引起的结构冲突



为消除结构冲突而插入的流水线气泡

引入暂停后的时空图

指令编号	时钟周期									
	1	2	3	4	5	6	7	8	9	10
指令i	IF	ID	EX	MEM	WB					
指令i+1		IF	ID	EX	MEM	WB				
指令i+2			IF	ID	EX	MEM	WB	WB		
指令i+3				stall	IF	ID	EX	MEM	WB	
指令i+4						IF	ID	EX	MEM	WB
指令i+5							IF	ID	EX	MEM

- 有时流水线设计者允许结构冲突的存在

  主要原因：减少硬件成本

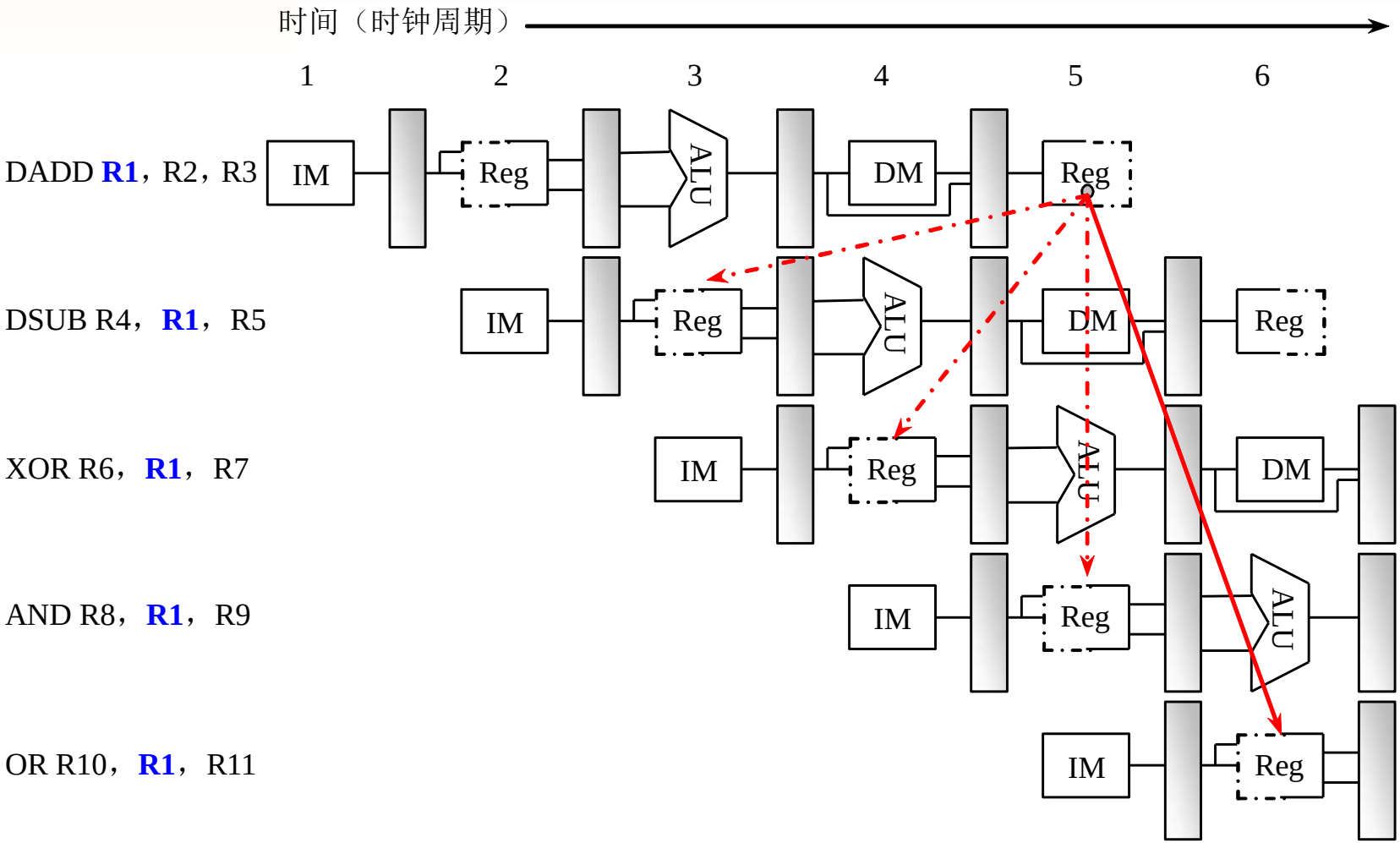
  - 如果把流水线中的所有功能单元完全流水化，或者重复设置足够份数，那么所花费的成本将相当高。
  - 有些结构冲突发生的概率不高

（2）数据冲突 （重点）

当相关的指令靠得足够近时，它们在流水线中的重叠执行或者重新排序会改变指令读/写操作数的顺序，使之不同于它们非流水实现时的顺序，则发生了数据冲突。

数据冲突举例：@24

```
DADD  R1，R2，R3
DSUB  R4，R1，R5
XOR   R6，R1，R7
AND   R8，R1，R9
OR    R10，R1，R11
```


时间（时钟周期）
1
2
3
4
5
6
DADD R1，R2，R3
DSUB R4，R1，R5
XOR R6，R1，R7
AND R8，R1，R9
OR R10，R1，R11
IM
Reg
ALU
DM

流水线的数据冲突举例

- 根据指令读访问和写访问的顺序，可以将数据冲突分为3种类型。

考虑两条指令*i*和*j*@25，且*i*在*j*之前进入流水线，可能发生的数据冲突有：

- 写后读冲突（RAW）

在 *i* 写入之前，*j* 先去读。
j 读出的内容是错误的。

这是最常见的一种数据冲突，它对应于真数据相关。

- 读后写冲突（WAR）

  在 *i* 读之前，*j* 先写。

  i 读出的内容是错误的！

由名相关里的反相关引起。

这种冲突仅发生在这样的情况下：

- 有些指令的写结果操作提前了，而且有些指令的读操作滞后了。
- 指令被重新排序了。

读后写冲突在前述5段流水线中不会发生。

（读操作（在ID段）在写结果操作（在WB段）之前）

- 写后写冲突（WAW）

  在 *i* 写入之前，*j* 先写。

  最后写入的结果是 *i* 的。错误！

  这种冲突对应于名相关里的输出相关。

写后写冲突仅发生在这样的流水线中：

- 流水线中不只一个段可以进行写操作。
- 当先前某条指令停顿时，允许其后续指令继续前进。

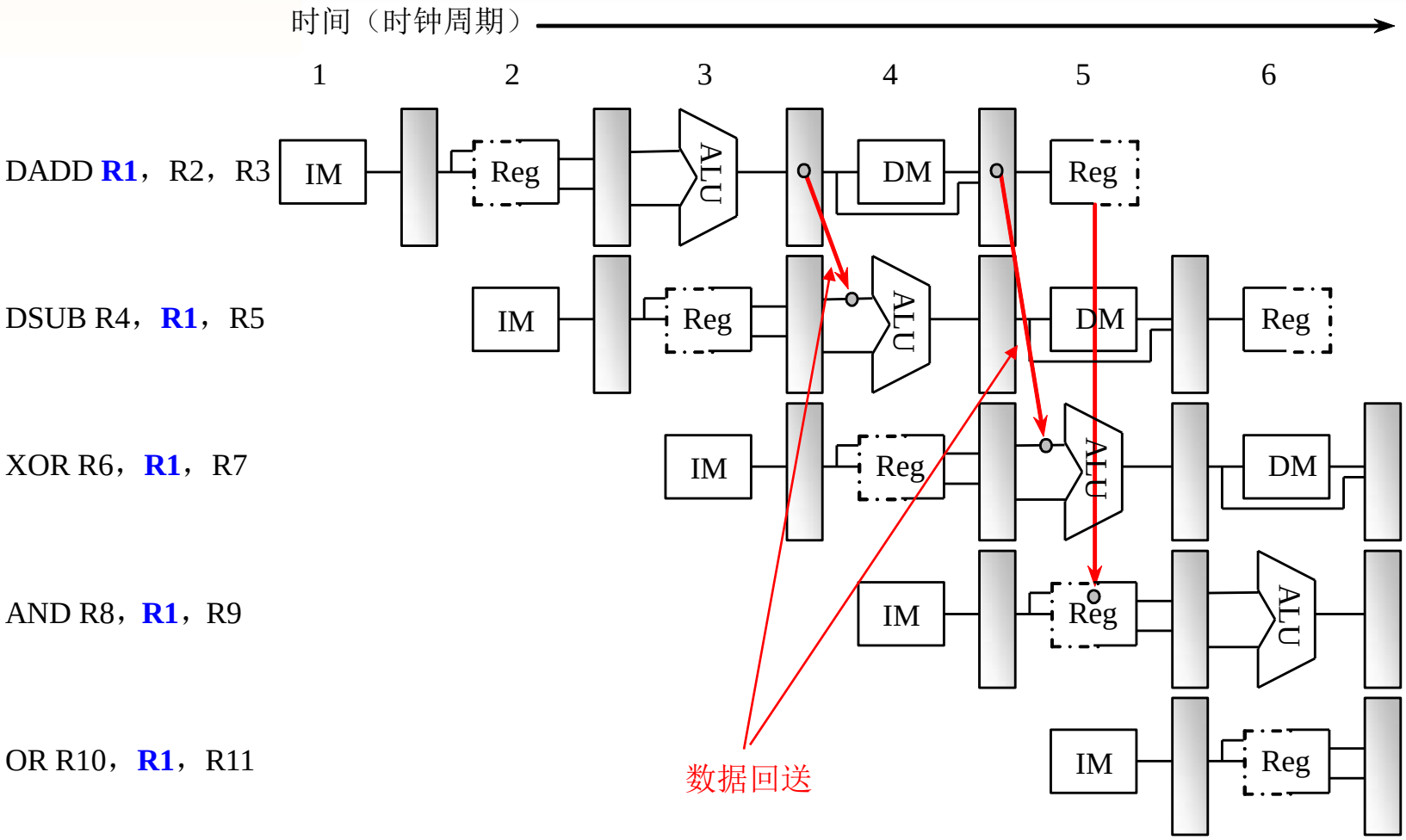
前面介绍的5段流水线不会发生写后写冲突。

（只在WB段写寄存器）

- 通过定向技术减少数据冲突引起的停顿

  （定向技术也称为旁路或短路）

  - 关键思想：在某条指令产生计算结果之前，其他指令并不真正立即需要该计算结果，如果能够将该计算结果从其产生的地方直接送到其他指令需要它的地方，那么就可以避免停顿。
  - 采用定向技术消除上例中的相关@26



采用定向技术后的流水线数据通路

- 当定向硬件检测到前面某条指令的结果寄存器就是当前指令的源寄存器时，控制逻辑会将前面那条指令的结果直接从其产生的地方定向到当前指令所需的位置。
- 结果数据不仅可以从某一功能部件的输出定向到其自身的输入，而且还可以定向到其他功能部件的输入。

举例:@27

```
DSUB  R1，R2，R3
LD    R5，0（R1）
SD    R5，12（R1）
```

- 需要停顿的数据冲突
  - 并不是所有的数据冲突都可以用定向技术来解决。

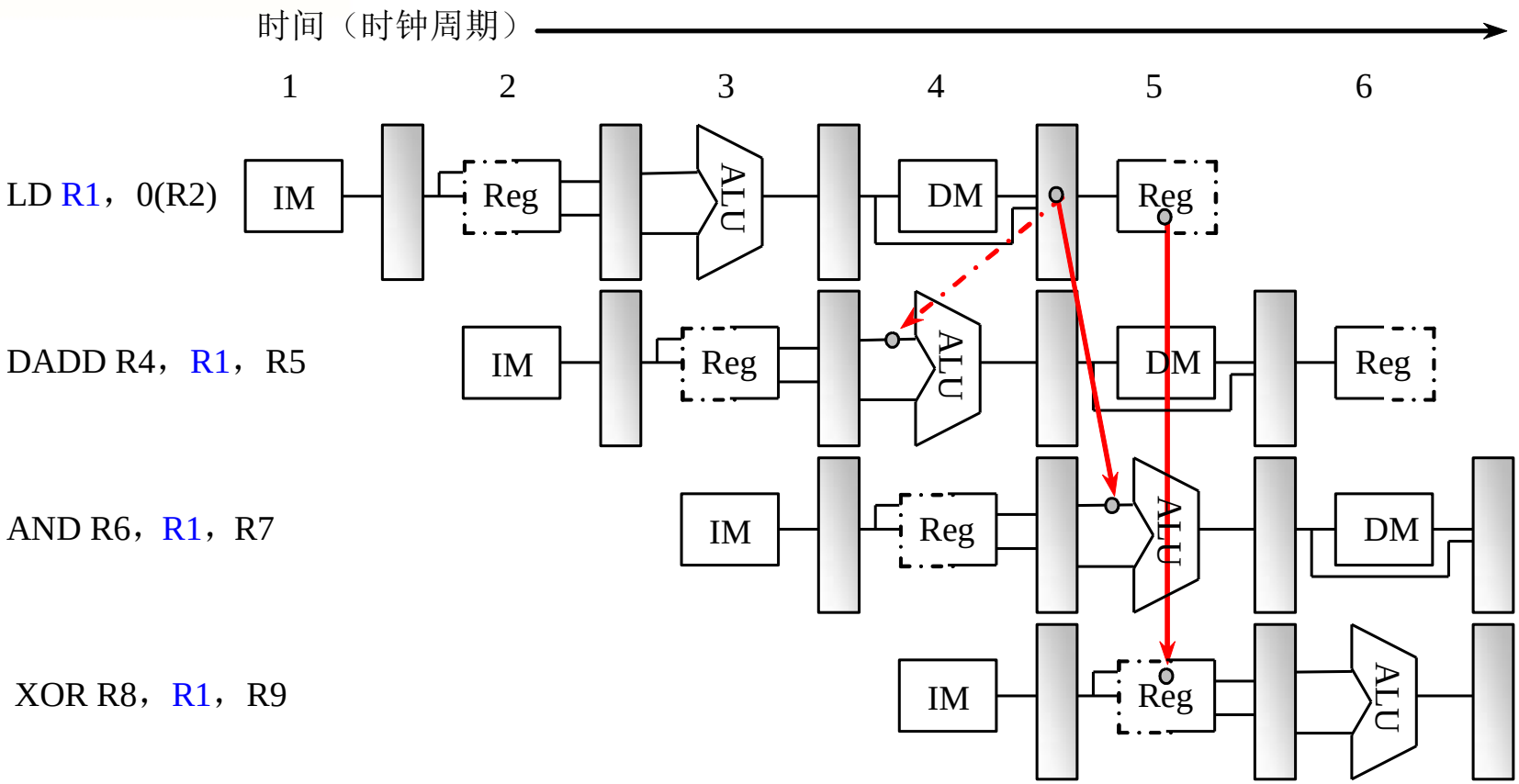
    举例:@28

```
LD    R1, 0 (R2)
DADD  R4, R1, R5
AND   R6, R1, R7
XOR   R8, R1, R9
```

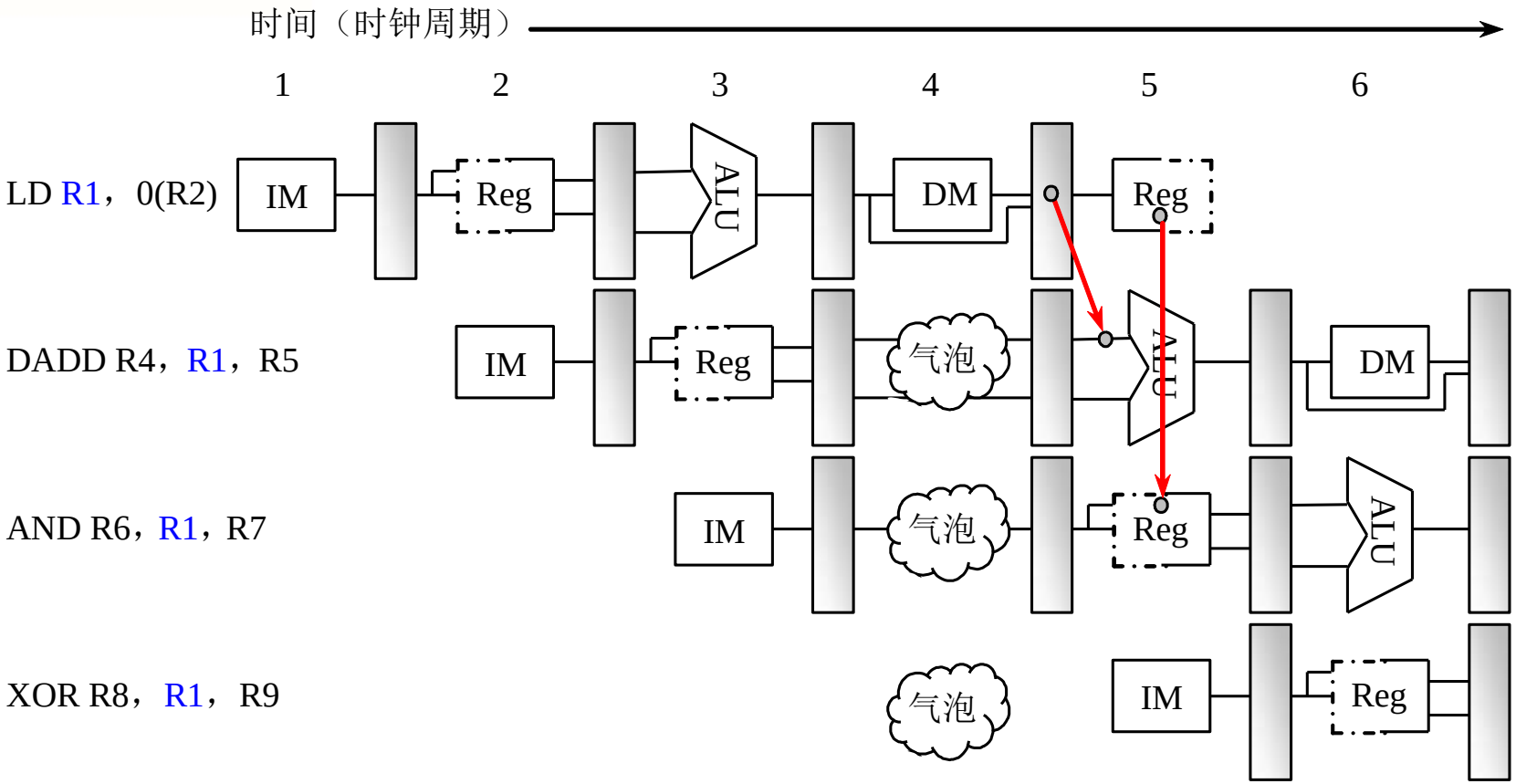
  - 增加流水线互锁（pipeline interlock）硬件，插入“暂停”。

    作用：检测发现数据冲突，并使流水线停顿@29，直至冲突消失。

    举例：演示A@30　　演示B@31



无法将LD指令的结果定向到DADD指令



流水线互锁机制插入气泡后的执行过程

LD R1，0（R2）	IF	ID	EX	MEM	WB			
DADD R4，R1，R5		IF	ID	EX	MEM	WB		
AND R6，R1，R7			IF	ID	EX	MEM	WB	
XOR R8，R1，R9				IF	ID	EX	MEM	WB

LD R1，0（R2）	IF	ID	EX	MEM	WB			
DADD R4，R1，R5		IF	ID	stall	EX	MEM	WB	
AND R6，R1，R7			IF	stall	ID	EX	MEM	WB
XOR R8，R1，R9				stall	IF	ID	EX	MEM

插入停顿前后的流水线时空图

总结：通过硬件检测到的各种冲突情况@32

- 依靠编译器解决数据冲突

  让编译器重新组织指令顺序来消除冲突，这种技术称为指令调度或流水线调度。

  - 例如：采用典型的代码生成方法，

    表达式A＝B＋C的代码会导致暂停@33

LD Rb，B	IF	ID	EX	MEM	WB				
LD Rc，C		IF	ID	EX	MEM	WB			
DADD Ra，Rb，Rc			IF	ID	stall	EX	MEM	WB	
SD Ra ，A				IF	stall	ID	EX	MEM	WB

- 举例：

请为下列表达式生成没有暂停的指令序列：

A＝B＋C ；

D＝E－F ；

假设载入延迟为1个时钟周期。

题解@34

调度前的代码		调度后的代码	
LD	Rb，B	LD	Rb，B
LD	Rc，C	LD	Rc，C
DADD	Ra，Rb，Rc	LD	Re，E
SD	Ra，A	DADD	Ra，Rb，Rc
LD	Re，E	LD	Rf，F
LD	Rf，F	SD	Ra，A
DSUB	Rd，Re，Rf	DSUB	Rd，Re，Rf
SD	Rd，D	SD	Rd，D

（3）控制冲突

- 执行分支指令的结果有两种
  - 分支成功：PC值改变为分支转移的目标地址。
    在条件判定和转移地址计算都完成后，才改变PC值(MEM段)。
  - 不成功或者失败：PC的值保持正常递增，
    指向顺序的下一条指令。
- 处理分支指令最简单的方法：
  “冻结”或者“排空”流水线 。
  优点：简单。
  - 前述5段流水线中，改变PC值是在MEM段进行的。
    给流水线带来了3个时钟周期的延迟@35。

简单处理分支指令：分支成功的情况

分支指令	IF	ID	EX	MEM	WB					
分支目标指令		IF	stall	stall	IF	ID	EX	MEM	WB	
分支目标指令+1						IF	ID	EX	MEM	WB
分支目标指令+2							IF	ID	EX	MEM
分支目标指令+3								IF	ID	EX

简单处理分支指令：分支失败的情况

分支指令	IF	ID	EX	MEM	WB					
分支后继指令		IF	stall	stall	IF	ID	EX	MEM	WB	
分支后继指令+1						IF	ID	EX	MEM	WB
分支后继指令+2							IF	ID	EX	MEM
分支后继指令+3								IF	ID	EX

重复取指

- 把由分支指令引起的延迟称为分支延迟。
- 分支指令在目标代码中出现的频度
  - 每3～4条指令就有一条是分支指令。

    假设：分支指令出现的频度是30%，

    流水线理想 CPI ＝ 1，

    分支延迟为3个时钟周期，

    那么：流水线的实际 CPI ＝ 1.9（1+0.3x3）。

如何减少分支延迟？

- 可采取两种措施来减少分支延迟。（硬件方法）
  - 在流水线中尽早判断出分支转移是否成功；
  - 尽早计算出分支目标地址。

  两个尽早必须同时采用，缺一不可。

下面的讨论中，我们假设：

这两步工作被提前到ID段完成，即分支指令是在ID段的末尾执行完成，所带来的分支延迟为一个时钟周期。

- 3种通过软件（编译器）来减少分支延迟的方法

  共同点：

  - 对分支的处理方法在程序的执行过程中始终是不变的，是静态的。
  - 要么总是预测分支成功，要么总是预测分支失败。

  - 预测分支失败
    - 允许分支指令后的指令继续在流水线中流动，就好象什么都没发生似的。
    - 若确定分支失败，将分支指令看作是一条普通指令，流水线正常流动。

- 若确定分支成功，流水线就把在分支指令之后取出的所有指令转化为空操作，并按分支目地重新取指令执行。

要保证：分支结果出来之前不会改变处理机的状态，以便一旦猜错时，处理机能够回退到原先的状态。

流水线的处理过程@36

流水线对分支的处理过程

分支指令 i(失败)	IF	ID	EX	MEM	WB				
指令 i+1		IF	ID	EX	MEM	WB			
指令 i+2			IF	ID	EX	MEM	WB		
指令 i+3				IF	ID	EX	MEM	WB	
指令 i+4					IF	ID	EX	MEM	WB

分支指令 i(成功)	IF	ID	EX	MEM	WB				
指令 i+1		IF	ID	idle	idle	idle			
分支目标 j			IF	ID	EX	MEM	WB		
分支目标 j+1				IF	ID	EX	MEM	WB	
分支目标 j+2					IF	ID	EX	MEM	WB

预测分支失败情况下的分支取消

- 预测分支成功

假设分支转移成功，并从分支目标地址处取指令执行。

前述5段流水线中，这种方法没有任何好处（因为是在同一时间段知道分支成功与否和得到目的地址的），该方法不是本书讨论重点。

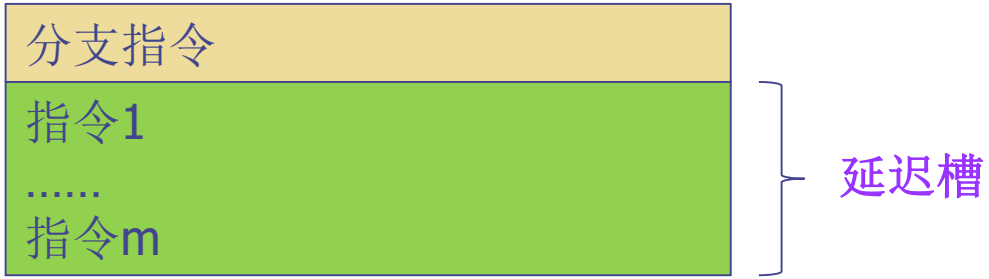
起作用的场合：先知道分支目标地址，后知道分支是否成功的系统中。

拓展: GCC的编译预处理likely()、unlikely()

- 延迟分支

主要思想：

在分支指令后面加上*m*条指令（延迟槽），把它们看成一个整体。不管分支是否成功，都顺序执行这些指令。



用延迟槽中的指令掩盖分支延迟。

具有一个分支延迟槽的流水线的执行过程@37

分支失败	分支指令i	IF	ID	EX	MEM	WB				
	延迟槽指令 i+1		IF	ID	EX	MEM	WB			
	指令 i+2			IF	ID	EX	MEM	WB		
	指令 i+3				IF	ID	EX	MEM	WB	
	指令 i+4					IF	ID	EX	MEM	WB

分支成功	分支指令i	IF	ID	EX	MEM	WB				
	延迟槽指令 i+1		IF	ID	EX	MEM	WB			
	分支目标指令j			IF	ID	EX	MEM	WB		
	分支目标指令j+1				IF	ID	EX	MEM	WB	
	分支目标指令j+2					IF	ID	EX	MEM	WB

分支延迟槽中的指令“掩盖”了流水线原来必须插入的暂停周期。

分支延迟指令的调度

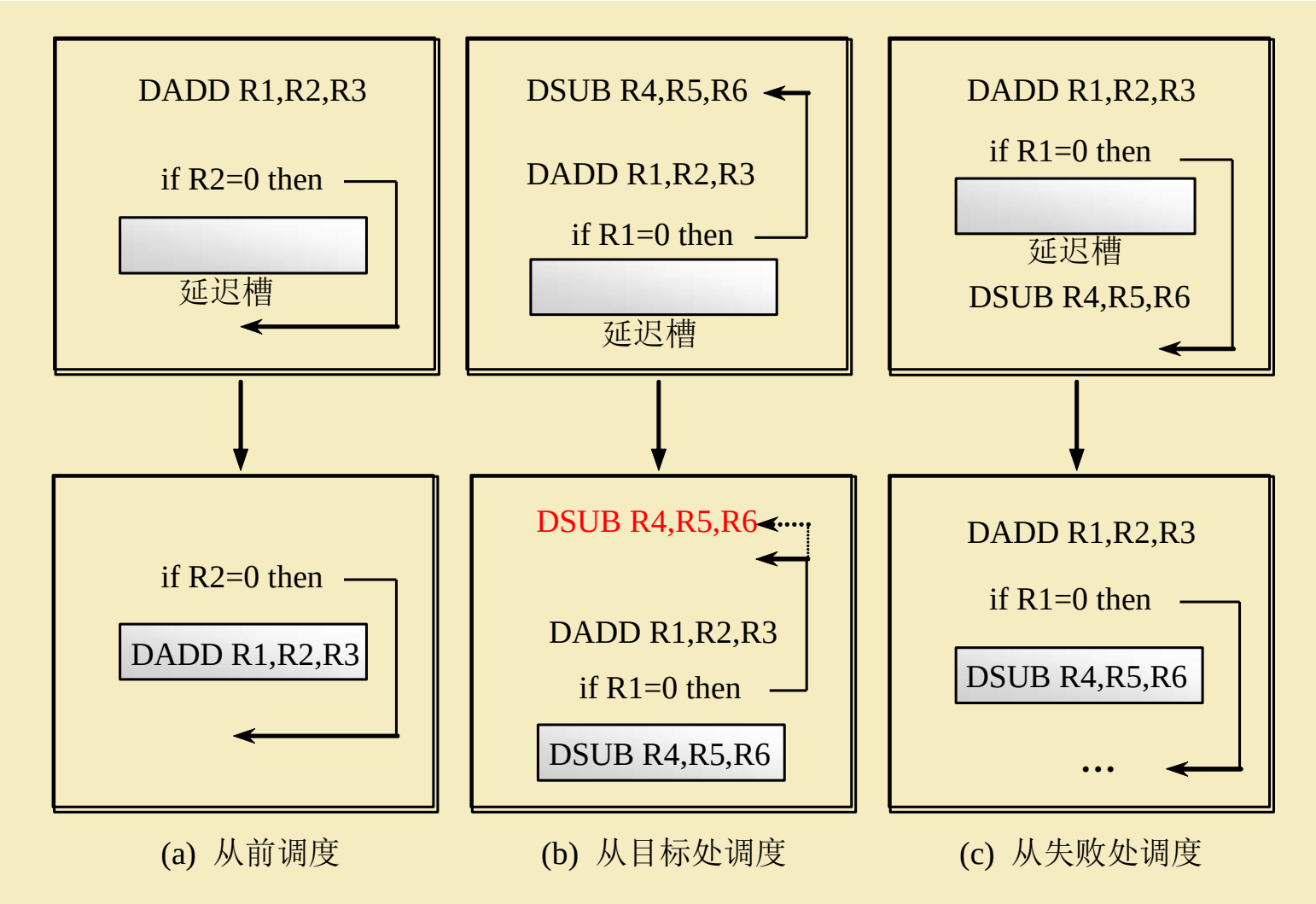
任务：在延迟槽中放入有用的指令。

由编译器完成。能否带来好处取决于编译器能否把有用的指令调度到延迟槽中。

三种调度方法：@38

- 从前调度
- 从目标处调度
- 从失败处调度

调度前和调度后的代码



(a) 从前调度　(b) 从目标处调度　(c) 从失败处调度

三种方法的要求及效果

调 度 策 略	对调度的要求	什么情况下起作用
从前调度（首选）	被调度的指令必须与分支无关	任何情况
从目标处调度	必须保证在分支失败时执行被调度的指令不会导致错误。有可能需要复制指令	分支成功时（但由于复制指令，有可能会增大程序空间）
从失败处调度	必须保证在分支成功时执行被调度的指令不会导致错误	分支失败时

分支延迟受到两个方面的限制：

- 可以被放入延迟槽中的指令要满足一定的条件。
- 编译器预测分支转移方向的能力。

进一步改进：分支取消机制（分支取消@39）

当分支的实际执行方向和事先所预测的一样时，执行分支延迟槽中的指令，否则就将分支延迟槽中的指令转化成一个空操作。(动态调度)

“预测成功－取消”分支的执行过程@40

分支失败	分支指令i	IF	ID	EX	MEM	WB				
	延迟槽指令 分支目标指令j		IF	idle	idle	idle	idle			
	指令 i+1			IF	ID	EX	MEM	WB		
	指令 i+2				IF	ID	EX	MEM	WB	
	指令 i+3					IF	ID	EX	MEM	WB

分支成功	分支指令i	IF	ID	EX	MEM	WB				
	延迟槽指令 分支目标指令j		IF	ID	EX	MEM	WB			
	分支目标指令j+1			IF	ID	EX	MEM	WB		
	分支目标指令j+2				IF	ID	EX	MEM	WB	
	分支目标指令j+3					IF	ID	EX	MEM	WB

预测分支成功(不适合3.4实例)的情况下，分支取消机制的执行情况

总结：

指令相关是流水线冲突的主要原因，而冲突又可能引起流水线停顿，所以我们要处理相关问题，而处理相关问题要针对相关进行分类：

1）有真数据流动的数据相关必须保持相关特性，但为了避免冲突可以通过指令调度（包括静态或动态）来解决，也可以通过旁路（改进流水线硬件）来消除或缩小由于数据相关引起的RAW冲突造成的流水线停顿；

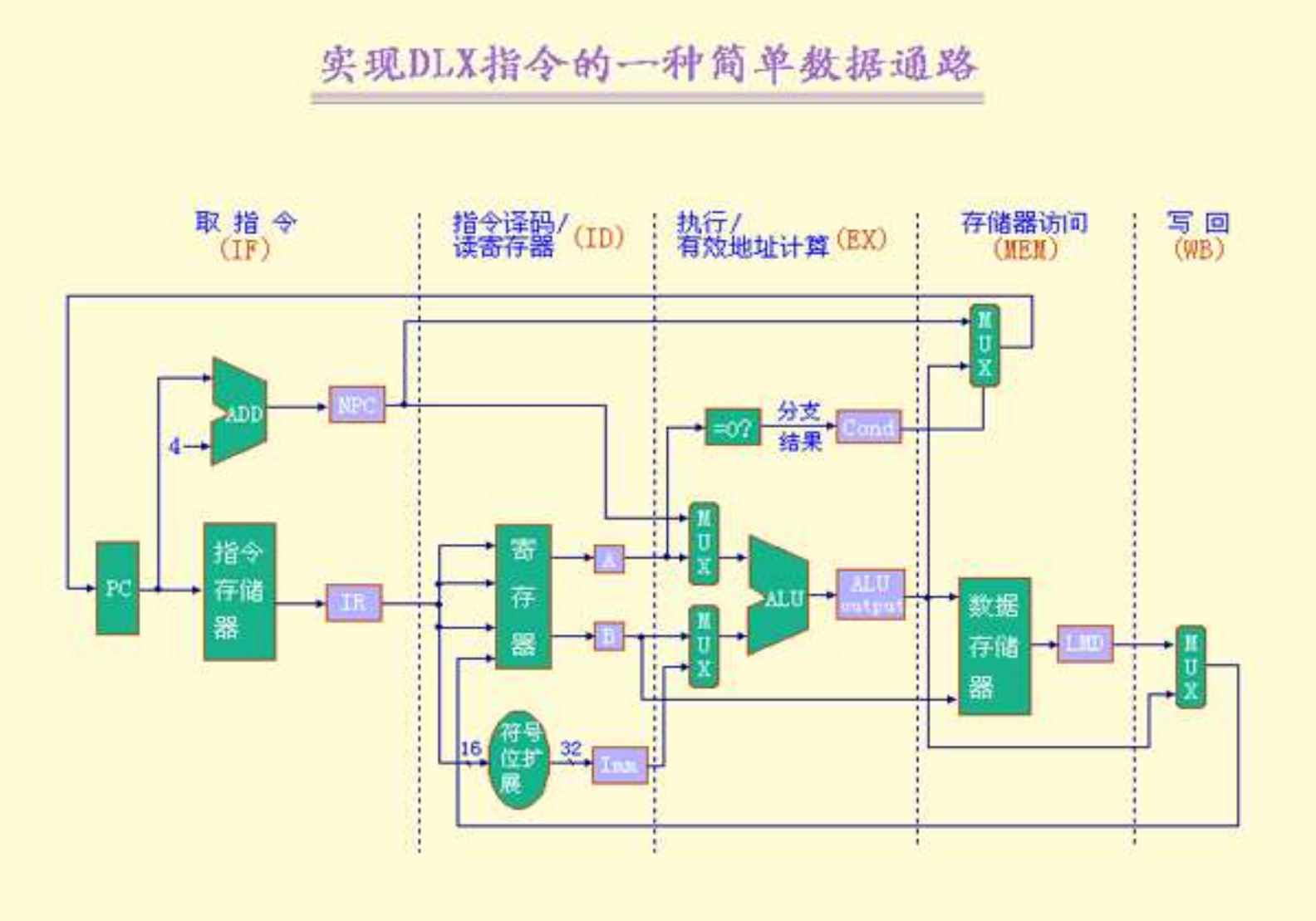
2）没有数据流动关系的名相关可以通过换名（使用其他的寄存器）消除相关，相关消除了，就不用再受指令执行顺序的制约，因名相关可能引起的WAR或WAW冲突自然也就不存在了，更不会引起流水停顿。

3.1 流水线的基本概念
3.2 流水线的性能指标
3.3 流水线的相关与冲突
3.4 **流水线的实现**
3.5 向量处理机

3.4 流水线的实现

3.4.1 MIPS的一种简单实现

1. 实现MIPS指令子集的一种简单数据通路@41。
   - 该数据通路的操作分成5个时钟周期
     - 取指令
     - 指令译码/读寄存器
     - 执行/有效地址计算
     - 存储器访问/分支完成
     - 写回
   - 只讨论整数指令的实现（包括：load和store，等于0转移，整数ALU指令等。）



NPC、IR、A、B、Imm、Cond、ALUo、LMD等临时寄存器是在单指令执行过程中保存中间结果的。

2. 一条MIPS指令最多需要以下5个时钟周期：

- 取指令周期（IF） 操作@42
  - IR←Mem[PC]
  - NPC←PC+4
- 指令译码/读寄存器周期（ID） 操作@43
  - A ← Regs[rs]
  - B ← Regs[rt]
  - Imm ← （（IR_{16}）^{16}##$IR_{16..31}$）

指令的译码操作和读寄存器操作是并行进行的。

原因：在MIPS指令格式中，操作码字段以及rs、rt字段都是在固定的位置。

这种技术称为固定字段译码技术。

- 执行/有效地址计算周期（EX）

不同指令所进行的操作不同：

- 存储器访问指令 操作@44

  ALUo←A + Imm

- 寄存器－寄存器ALU指令 操作@45

  ALUo←A func B

- 寄存器－立即值ALU指令 操作@46

  ALUo←A op Imm

- 分支指令 操作@47

  ALUo←NPC+（Imm<<2）；

  cond←（A == 0）

将有效地址计算周期和执行周期合并为一个时钟周期，这是因为MIPS指令集采用load / store结构，没有任何指令需要同时进行数据有效地址的计算、转移目标地址的计算和对数据进行运算。

- 存储器访问/分支完成周期（MEM）
  - 所有指令都要在该周期对PC进行更新。
    除了分支指令，其他指令都是做PC←NPC
  - 在该周期内处理的MIPS指令仅仅有load、store和分支三种指令。

- 存储器访问指令 操作@48

  LMD←Mem[ALUo]

  或者 Mem[ALUo]←B

- 分支指令 操作@49

  if （cond） PC ←ALUo else PC←NPC

➢ 写回周期（WB）

不同的指令在写回周期完成的工作也不一样。

- 寄存器－寄存器ALU指令 操作@50

  Regs[rd]← ALUo

- 寄存器－立即数ALU指令 操作@51

  Regs[rt]← ALUo

- load指令 操作@52

  Regs[rt]← LMD

3. 不采用单周期实现方案的主要原因

- 对于大多数CPU来说，单周期实现效率很低，因为不同的指令所需完成的操作差别相当大，因而所需要的时钟周期时间也大不一样。
- 单周期实现时，需要重复设置某些功能部件，而在多周期实现方案中，这些部件是可以共享的。

3.4.2 基本的MIPS流水线

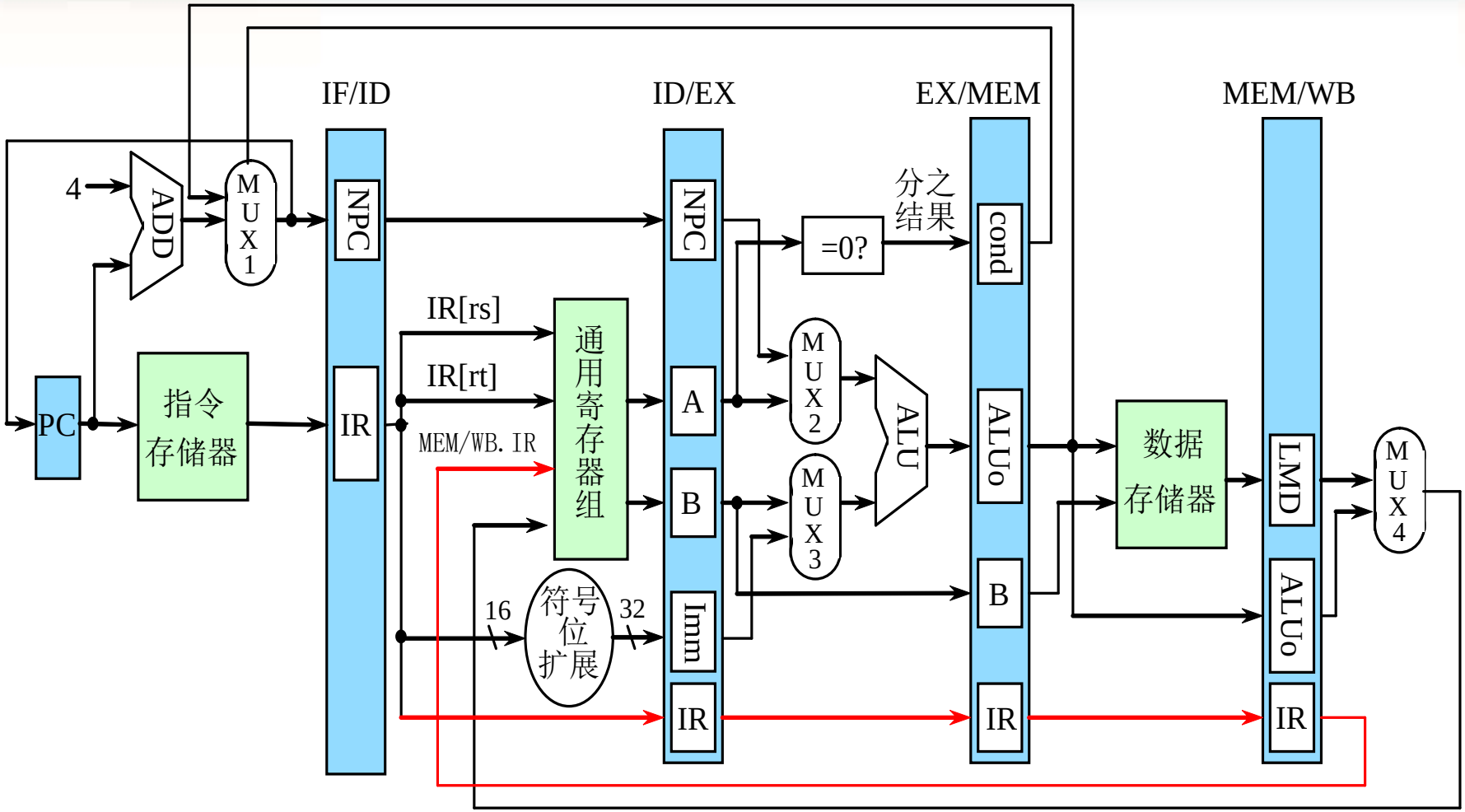
- 每一个时钟周期完成的工作看作是流水线的一段，每个时钟周期启动一条新的指令。

1. 流水实现的数据通路（改动）

- 设置了流水寄存器@53
  - 段与段之间设置流水寄存器
  - 流水寄存器的命名 作用@54

    用其相邻的两个段的名称拼合而成。

    例如：ID段与EX段之间的流水寄存器用ID/EX表示
  - 每个流水寄存器是由若干个寄存器构成的 @55


IF/ID
ID/EX
EX/MEM
MEM/WB
4
ADD
MUX1
NPC
IR
PC
指令存储器
IR[rs]
IR[rt]
MEM/WB. IR
通用寄存器组
16
符号位扩展
32
NPC
A
B
Imm
IR
=0?
分之结果
MUX2
MUX3
ALU
cond
ALUo
B
IR
数据存储器
LMD
ALUo
IR
MUX4

流水实现的数据通路

- 寄存器的命名形式为：x.y
- 所包含的字段的命名形式为：x.y[s]

  其中：x：流水寄存器名称

  y：具体寄存器名称

  s：字段名称

  例如：

  ID/EX.IR：流水寄存器ID/EX中的子寄存器IR

  IRID/EX.IR[op]：该寄存器的op字段（即操作码字段）
- 流水寄存器的作用
  - 将各段的工作隔开，使得它们不会互相干扰。
  - 保存相应段的处理结果。

例如：

EX/MEM.ALUo：保存EX段ALU的运算结果

MEM/WB.LMD：保存MEM段从数据存储器读出的数据

- 向后传递后面将要用到的数据或者控制信息

  所有有用的数据和控制信息每个时钟周期会随着指令在流水线中的流动往后流动一段。

- 增加了向后传递IR和从MEM/WB.IR回送到通用寄存器组的连接。
- 将对PC的修改移到了IF段，以便PC能及时地加4，为取下一条指令做好准备。

2. 每一个流水段进行的操作

- $IR[rs]=IR_{6..10}$
- $IR[rt]=IR_{11..15}$
- $IR[rd]=IR_{16..20}$

流水线的每个流水段的操作

流水段	所有指令类型		
IF	IF/ID.IR ← Mem[PC] IF/ID.NPC, PC ← (if ((EX/MEM.IR[op] == branch) & EX/MEM.cond) {EX/MEM.ALUo} else {PC+4}); (动画演示@56)		
ID	ID/EX.A ← Regs[IF/ID.IR[rs]]; ID/EX.B ← Regs[IF/ID.IR[rt]]; ID/EX.NPC ← IF/ID.NPC; ID/EX.IR ←IF/ID.IR; ID/EX.Imm ← $(IF/ID.IR_{16})^{16}$##$IF/ID.IR_{16..31}$; (动画演示@57)		
	ALU 指令	load/store 指令	分支指令
EX	EX/MEM.IR ← ID/EX.IR; EX/MEM.ALUo ← ID/EX.A *func* ID/EX.B 或 EX/MEM.ALUo ← ID/EX.A *op* ID/EX.Imm; (动画演示@58)	EX/MEM.IR ← ID/EX.IR; EX/MEM.ALUo ← ID/EX.A + ID/EX.Imm; EX/MEM.B←ID/EX.B; (动画演示@59)	EX/MEM.IR ← ID/EX.IR; EX/MEM.ALUo ← ID/EX.NPC + ID/EX.Imm<<2; EX/MEM.cond ← (ID/EX.A ==0); (动画演示@60)

流水线的每个流水段的操作

流水段	任何指令类型		
	ALU 指令	load/store 指令	分支指令
MEM	MEM/WB.IR ←EX/MEM.IR; MEM/WB.ALUo ← EX/MEM.ALUo; (动画演示@61)	MEM/WB.IR ← EX/MEM.IR; MEM/WB.LMD ← Mem[EX/MEM.ALUo]; 或 Mem[EX/MEM.ALUo] ← EX/MEM.B; (动画演示@62)	
WB	Regs[MEM/WB.IR[rd]] ← MEM/WB.ALUo; 或 Regs[MEM/WB.IR[rt]] ← MEM/WB.ALUo; (动画演示@63)	Regs[MEM/WB.IR[rt]] ← MEM/WB.LMD; (动画演示@64)	

3. 流水线的控制

- 主要是如何控制4个多路选择器@65。
  - MUX2：若ID/EX.IR中的指令是分支指令，则选择ID/EX.NPC，否则选ID/EX.A。
  - MUX3：若ID/EX.IR中的指令是寄存器－寄存器型ALU指令，则选ID/EX.B，否则选ID/EX.Imm。
  - MUX1：若EX/MEM.IR中的指令是分支指令，而且EX/MEM.cond为真，则选EX/MEM.ALUo，即分支目标地址，否则选PC+4。
  - MUX4：若MEM/WB.IR中的指令是load指令，则选MEM/WB.LMD，否则选MEM/WB.ALUo。

- 第5个多路器（图中未画）：从MEM/WB回传至通用寄存器组的写入地址应该是从MEM/WB.IR[rd] 和MEM/WB.IR[rt]中选一个。
  - 寄存器－寄存器型ALU指令：选择MEM/WB.IR[rd]；
  - 寄存器－立即数型ALU指令和load指令：选择MEM/WB.IR[rt]。

➢ 解决数据冲突的问题

- 所有的数据冲突均可以在ID段检测到。

  如果存在数据冲突，就在相应的指令流出ID段之前将之暂停。

  完成该工作的硬件称为流水线的互锁机制。

- 若采用定向技术，可在ID段确定需要什么样的定向，并设置相应的控制。

  降低流水线的硬件复杂度，不必挂起已经改变了机器状态的指令。

- 也可以在使用操作数的那个时钟周期（**EX**和**MEM**段）的开始检测冲突和确定必需的定向。
- 检测冲突是通过比较寄存器地址是否相等来实现的。

举例： load互锁

由于使用load指令而引起的流水线互锁称为load互锁。

如：ID段检测由于使用load指令导致RAW冲突@66
（这时load指令在EX段）

ID/EX中的操作码（ID/EX.IR[op]）	IF/ID中的操作码（IF/ID.IR[op]）	比较的操作数字段
load	RR ALU	ID/EX.IR[rt]=IF/ID.IR[rs]
load	RR ALU	ID/EX.IR[rt]=IF/ID.IR[rt]
load	load、store ALU立即数或分支	ID/EX.IR[rt]=IF/ID.IR[rs]

- 若检测到RAW冲突，流水线互锁机制必须在流水线中插入停顿，并使当前正处于IF段和ID段的指令不再前进。
  - 将ID/EX.IR中的操作码改为全0
    （全0表示空操作）
  - IF/ID寄存器的内容回送到自己的入口

➢ 定向逻辑

- 要考虑的情况更多
- 通过比较流水寄存器中的寄存器地址来确定

例如：

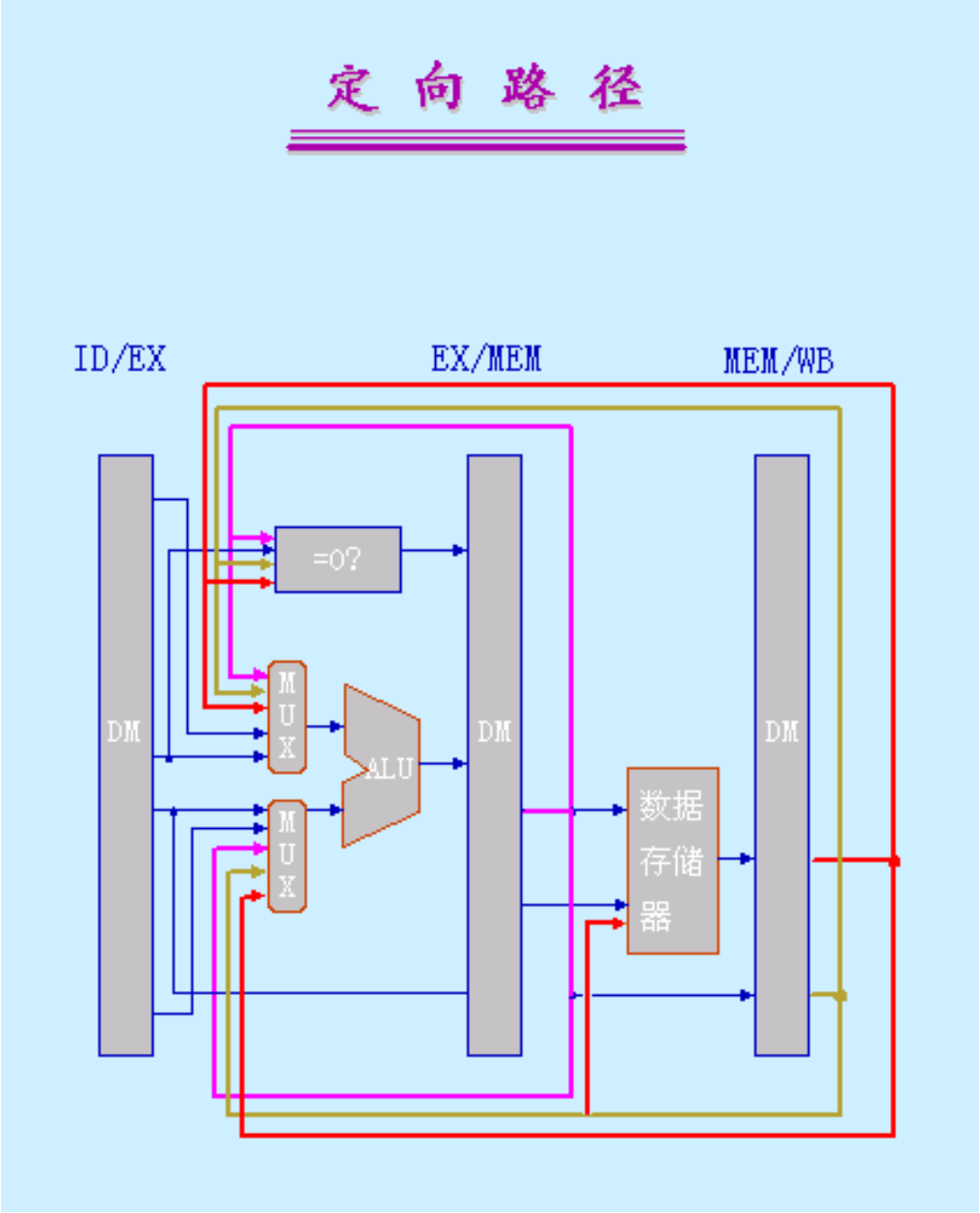
- 若（ID/EX.IR.op==RR ALU)&（EX/MEM.IR.op==RR ALU）&（ID/EX.IR[rt]==EX/MEM.IR[rd]），

  则 EX/MEM.ALUo定向到ALU的下面一个输入。

- 若（ID/EX.IR[op]==RR ALU）&（MEM/WB.IR[op]==load）&（ID/EX.IR[rt]==MEM/WB.IR[rt]），

  则 把MEM/WB.LMD定向到ALU的下面一个输入。

- 流水线增设的定向路径@67



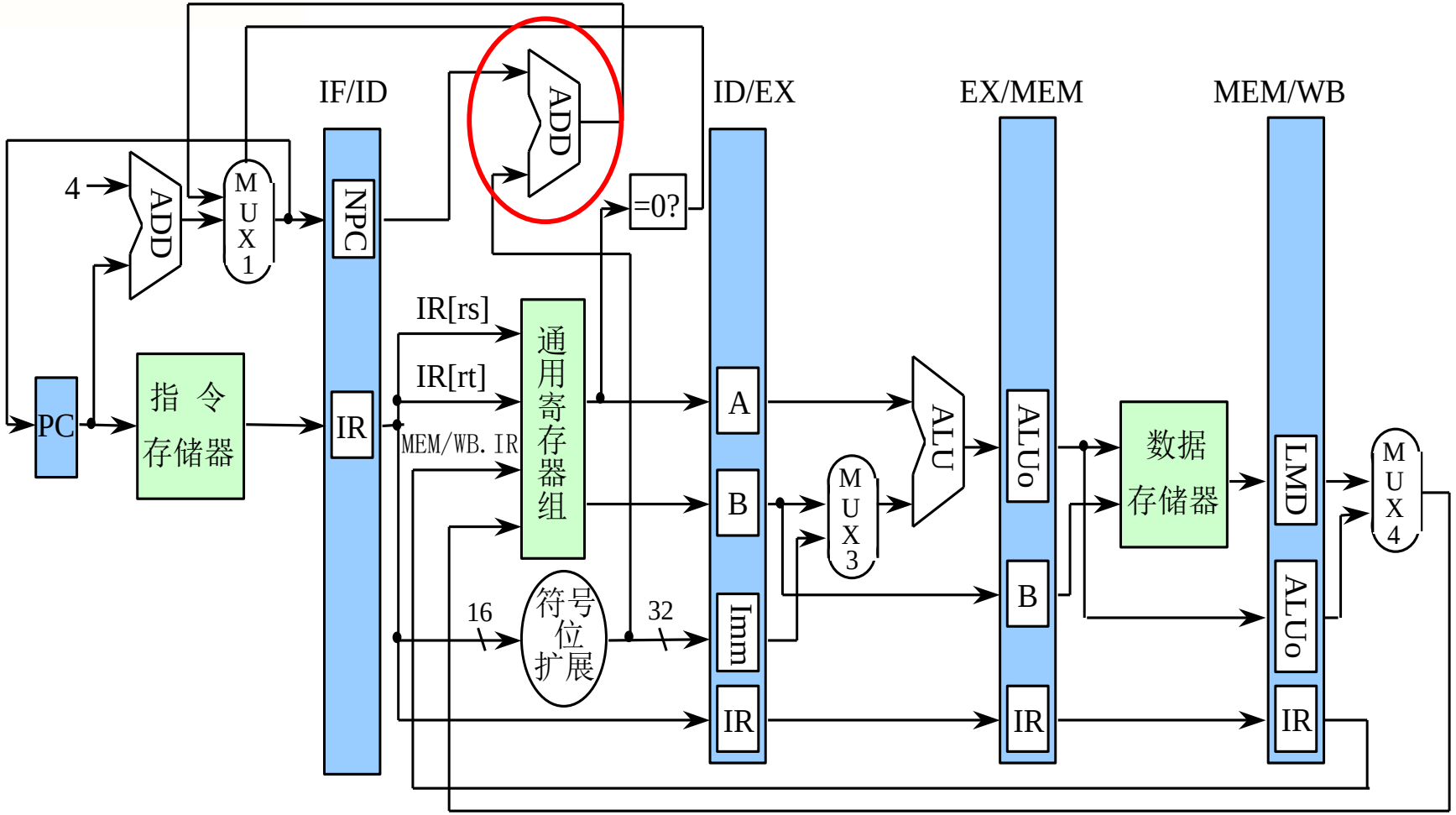
4. 控制冲突

- 分支指令的条件测试和分支目标地址计算在EX段完成，对PC的修改在MEM段完成@68。
- 它所带来的分支延迟是3个时钟周期。
- 减少分支延迟：

  （把上述工作提前到ID段进行）

  - 在ID段增设一个加法器，用于计算分支目标地址。
  - 把条件测试“=0？”的逻辑电路移到ID段。
  - 这些结果直接回送到IF段的MUX1。
  - 改进后的流水线对分支指令的处理。

3.4 流水线的实现


IF/ID
ID/EX
EX/MEM
MEM/WB
4
ADD
MUX 1
NPC
IR
PC
指 令
存储器
ADD
=0?
IR[rs]
IR[rt]
MEM/WB. IR
通用寄存器组
16
符号位扩展
32
A
B
Imm
IR
MUX 3
ALU
ALUo
B
IR
数据
存储器
LMD
ALUo
IR
MUX 4

改进后流水线的分支操作

流 水 段	分 支 指 令 操 作
IF	IF/ID.IR ← Mem[PC]; IF/ID.NPC, PC ← (if((IF/ID.IR[op]= =branch)&((Regs[IF/ID.IR[rs]] = = 0)) {IF/ID.NPC + (IF/ID.IR_{16})^{16} ## (IF/ID.$IR_{16..31}$ << 2) } else {PC+4});
ID	ID/EX.A ←Regs[IF/ID.IR[rs]]; ID/EX.B← Regs[IF/ID.IR[rt]]; ID/EX.IR ← IF/ID.IR; ID/EX.Imm ← (IF/ID.IR_{16})^{16} ## IF/ID. $IR_{16..31}$;
EX	
MEM	
WB	

3.1 流水线的基本概念
3.2 流水线的性能指标
3.3 流水线的相关与冲突
3.4 流水线的实现
3.5 **向量处理机**

3.5 向量处理机

- 在流水线处理机中，设置向量数据表示和相应的向量指令，称为向量处理机。
- 不具有向量数据表示和相应的向量指令的流水线处理机，称为标量处理机。

3.5.1 向量处理方式

以计算表达式 $D=A\times(B+C)$ 为例

A、B、C、D @69—— 长度为N的向量

1. 横向(水平)处理方式@70

- 向量计算是按行的方式从左到右横向地进行。
  - 先计算： $d_1 \leftarrow a_1 \times (b_1 + c_1)$
  - 再计算： $d_2 \leftarrow a_2 \times (b_2 + c_2)$
  - ……
  - 最后计算： $d_N \leftarrow a_N \times (b_N + c_N)$
- 组成循环程序进行处理。

$$k_i \leftarrow b_i + c_i$$

$$d_i \leftarrow k_i \times a_i$$

  - 数据相关：*N*次　功能切换：2*N*次
- 不适合于向量处理机的并行处理。

2. 纵向 (垂直)处理方式@71

- 向量计算是按列的方式从上到下纵向地进行。

$$\text{先计算}\begin{cases} k_1 \leftarrow b_1 + c_1 \\ \cdots\cdots \\ k_N \leftarrow b_N + c_N \end{cases} \qquad \text{再计算}\begin{cases} d_1 \leftarrow k_1 \times a_1 \\ \cdots\cdots \\ d_N \leftarrow k_N \times a_N \end{cases}$$

- 表示成向量指令：

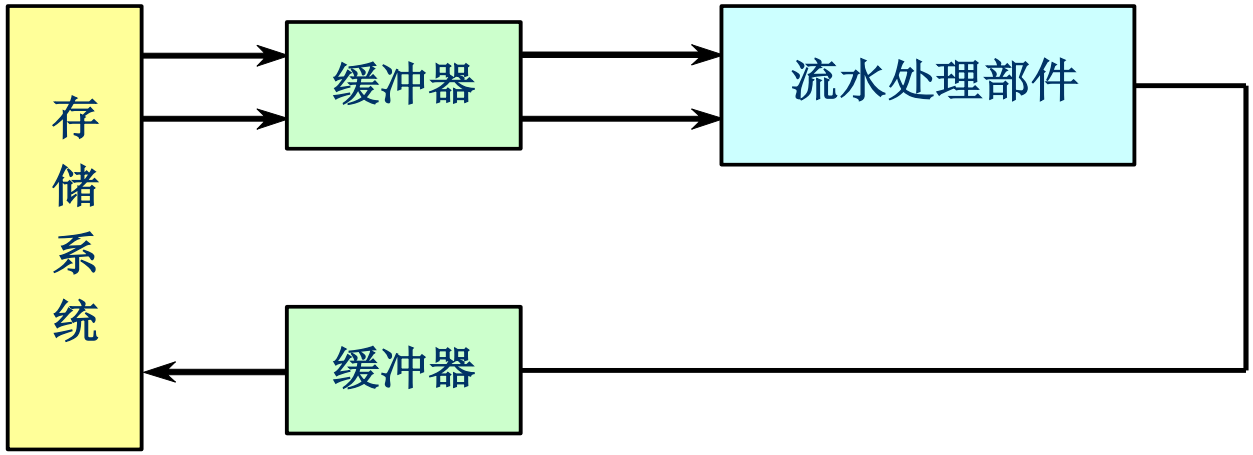
$$K = B + C$$

$$D = K \times A$$

  - 两条向量指令之间：

    数据相关：1次　　功能切换：1次

- 对处理机结构的要求：存储器—存储器结构
  - 向量指令的源向量和目的向量都存放在存储器中，运算的中间结果需要送回存储器。
  - 存储器—存储器型操作的运算流水线@72
  - 例如：STAR-100、CYBER-205



3. 纵横 (分组)处理方式@73

- 又称为分组处理方式。
- 把向量分成若干组，组内按纵向方式处理，依次处理各组。
- 对于上述的例子，设：

$$N=S\times n+r$$

  - 其中N为向量长度，S为组数，n为每组的长度，r为余数。
  - 若余下的r个数也作为一组处理，则共有$S+1$组。
- 运算过程为：

- 先算第1组：

$$k_{1\sim n} \leftarrow b_{1\sim n} + c_{1\sim n}$$

$$d_{1\sim n} \leftarrow k_{1\sim n} \times a_{1\sim n}$$

- 再算第2组：

$$k_{(n+1)\sim 2n} \leftarrow b_{(n+1)\sim 2n} + c_{(n+1)\sim 2n}$$

$$d_{(n+1)\sim 2n} \leftarrow k_{(n+1)\sim 2n} \times a_{(n+1)\sim 2n}$$

- 依次进行下去，直到最后一组：第S+1组。
- 每组内各用两条向量指令。

数据相关：1次　　功能切换：2次

- 对处理机结构的要求：寄存器－寄存器结构
  - 设置能快速访问的向量寄存器，用于存放源向量、目的向量及中间结果，让运算部件的输入、输出端都与向量寄存器相联，构成寄存器－寄存器型操作的运算流水线。
  - 典型的寄存器－寄存器结构的向量处理机

    美国的CRAY-1、我国的YH-1巨型机

3.5.2 向量处理机的结构

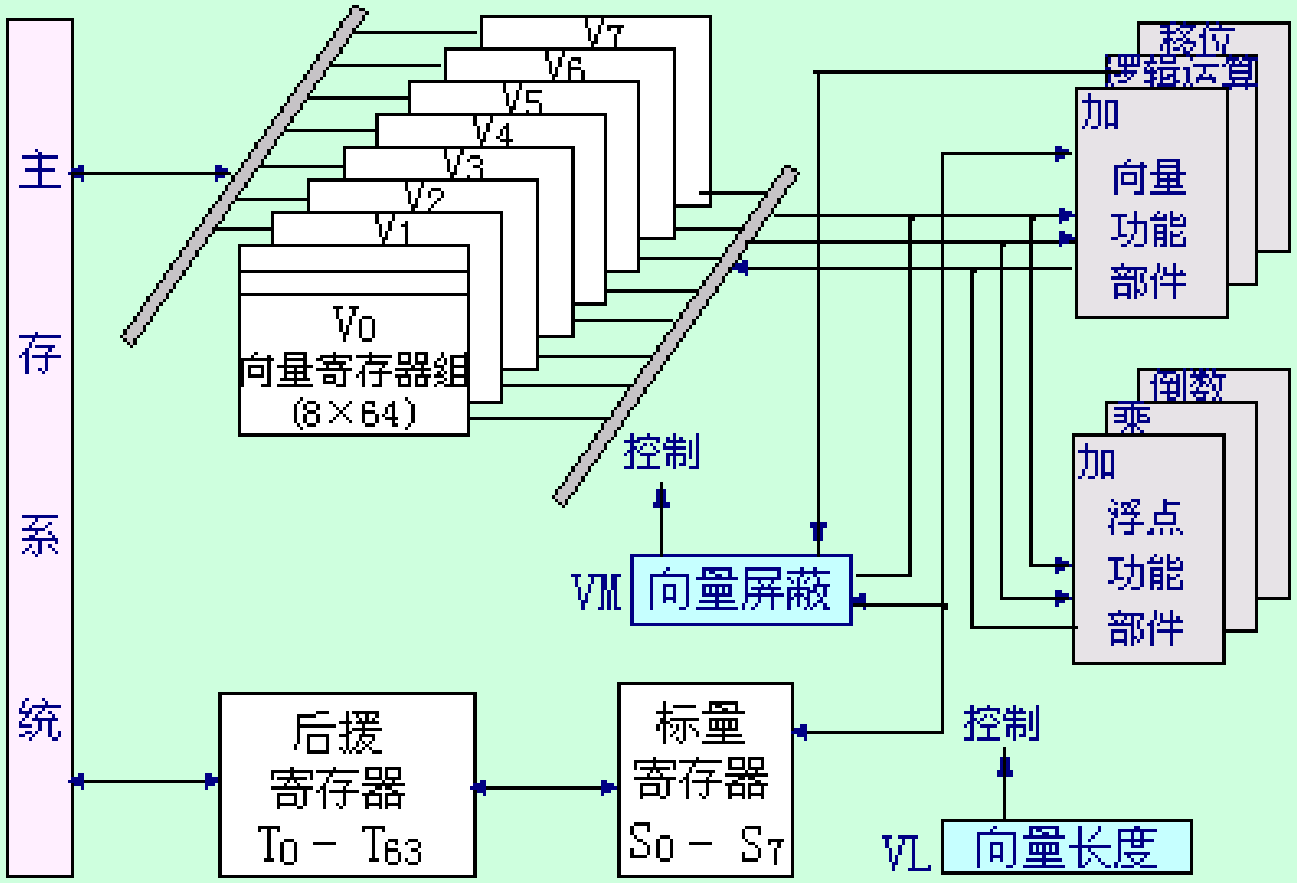
- 以CRAY-1机为例
  - 美国CRAY公司
  - 1976年
  - 每秒1亿次浮点运算
  - 时钟周期：12.5 ns

1. CRAY-1的基本结构@74

- 功能部件

共有12条可并行工作的单功能流水线，可分别流水地进行地址、向量、标量的各种运算。

CRAY-1的基本结构


主存系统
V7
V6
V5
V4
V3
V2
V1
V0
向量寄存器组
(8×64)
移位
逻辑运算
加
向量
功能
部件
倒数
乘
加
浮点
功能
部件
控制
VM
向量屏蔽
后援
寄存器
T0 - T63
标量
寄存器
S0 - S7
控制
VL
向量长度

- 6个单功能流水部件：进行向量运算
  - 整数加（3拍）
  - 逻辑运算（2拍）
  - 移位（4拍）
  - 浮点加（6拍）
  - 浮点乘（7拍）
  - 浮点迭代求倒数（14拍）

括号中的数字为其流水经过的时间，每拍为一个时钟周期，即12.5 ns。

- 向量寄存组V
  - 由512个64位的寄存器组成，分成8块。
  - 编号：V_0～V_7
  - 每一个块称为一个向量寄存器，可存放一个长度（即元素个数）不超过64的向量。
  - 每个向量寄存器可以每拍向功能部件提供一个数据元素，或者每拍接收一个从功能部件来的结果元素。
- 标量寄存器S和快速暂存器T
  - 标量寄存器有8个：S_0～S_7　64位
  - 快速暂存器T用于在标量寄存器和存储器之间提供缓冲。

- 向量屏蔽寄存器VM
  - 64位，每一位对应于向量寄存器的一个单元。
  - 作用：用于向量的归并、压缩、还原和测试操作、对向量某些元素的单独运算等。

2. CRAY-1向量处理的一个显著特点

- 每个向量寄存器V_i都有连到6个向量功能部件的单独总线。
- 每个向量功能部件也都有把运算结果送回向量寄存器组的总线。

- 只要不出现V_i冲突和功能部件冲突，各V_i之间和各功能部件之间都能并行工作，大大加快了向量指令的处理。
  - V_i冲突：并行工作的各向量指令的源向量或结果向量使用了相同的V_i。

例如：源向量相同

$$V_3 \leftarrow V_1 + V_2$$

$$V_5 \leftarrow V_4 \wedge V_1$$

  - 功能部件冲突：并行工作的各向量指令要使用同一个功能部件。

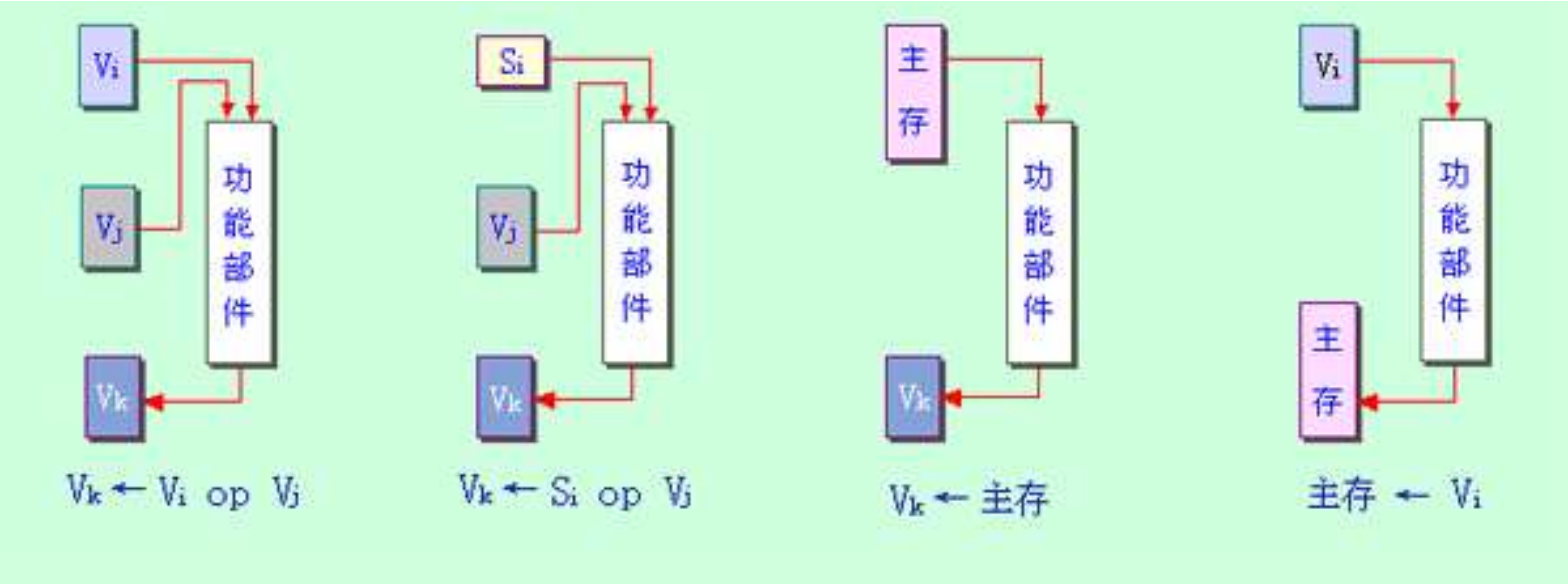
例如：都需使用乘法功能部件

$$V_3 \leftarrow V_1 \times V_2$$

$$V_5 \leftarrow V_4 \times V_6$$

3. CRAY-1向量指令类型

- $V_k \leftarrow V_i \text{ op } V_j$
- $V_k \leftarrow S_i \text{ op } V_j$
- $V_k \leftarrow$ 主存
- 主存 $\leftarrow V_i$



3.5.3 提高向量处理机性能的方法

提高向量处理机性能的方法

- 设置多个功能部件，使它们并行工作。
- 采用链接技术，加快一串向量指令的执行。
- 采用循环开采技术，加快循环的处理。
- 采用多处理机系统，进一步提高性能。

1. 设置多个功能部件

- 设置多个独立的功能部件。这些部件能并行工作，并各自按流水方式工作，从而形成了多条并行工作的运算操作流水线。

  例如：CRAY-1向量处理机有4组12个单功能流水部件：

  - 向量部件：向量加，移位，逻辑运算
  - 浮点部件：浮点加，浮点乘，浮点求倒数
  - 标量部件：标量加，移位，逻辑运算，数“1”/计数
  - 地址运算部件：整数加，整数乘

2. 链接技术

- 链接特征：具有先写后读相关的两条指令，在不出现功能部件冲突和源向量冲突的情况下，可以把功能部件链接起来进行流水处理，以达到加快执行的目的。
- 链接特性的实质

把流水线定向的思想引入到向量执行过程的结果。

例3.3 在CRAY-1上用链接技术进行向量运算

$$D=A\times(B+C)$$

假设向量长度$N\leqslant 64$，向量元素为浮点数，且向量B、C已存放在V_0和V_1中。

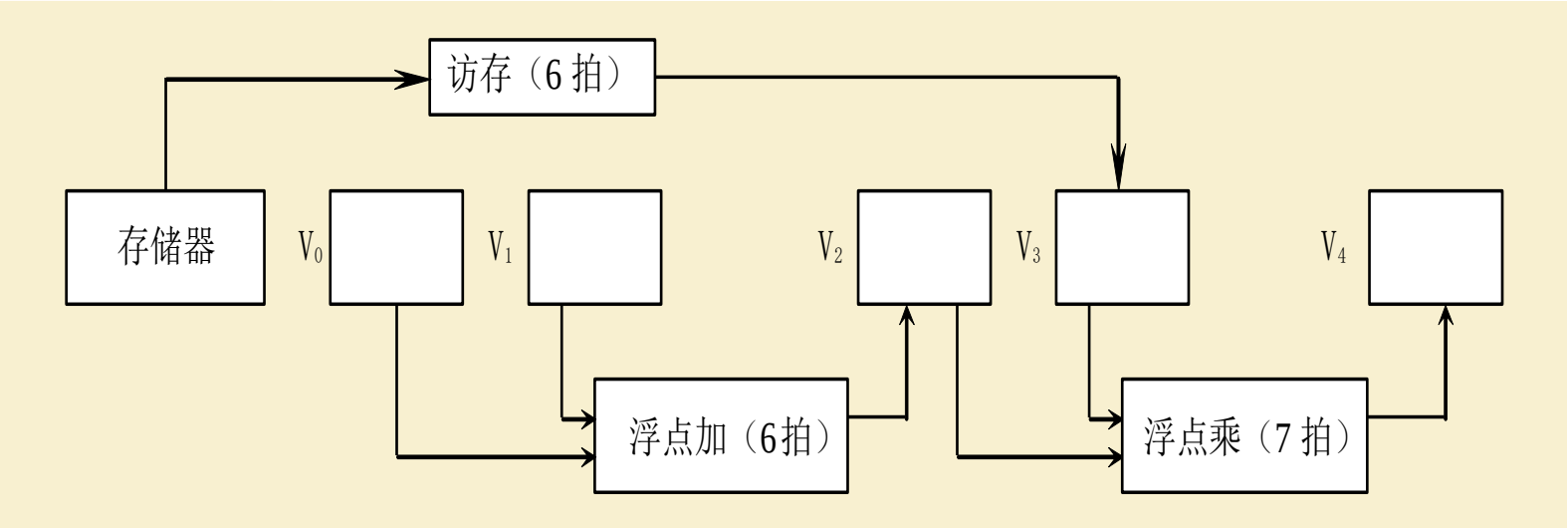
画出链接示意图，并分析非链接执行和链接执行两种情况下的执行时间。

解 用以下三条向量完成上述运算：

$V_3 \leftarrow$ 存储器 // 访存取向量A

$V_2 \leftarrow V_0 + V_1$ // 向量B和向量C进行浮点加

$V_4 \leftarrow V_2 \times V_3$ // 浮点乘，结果存入V_4



- 假设：把向量数据元素送往向量功能部件以及把结果存入向量寄存器需要一拍时间，从存储器中把数据送入访存功能部件需要一拍时间。

- 3条指令全部用串行方法执行，则执行时间为：

$$[(1+6+1)+N-1]+[(1+6+1)+N-1]+[(1+7+1)+N-1]=3N+22\ (拍)$$

- 前两条指令并行执行，然后再串行执行第3条指令，则执行时间为：

$$[(1+6+1)+N-1]+[(1+7+1)+N-1]=2N+15\ (拍)$$

- 第1、2条向量指令并行执行，并与第3条指令链接执行。

- 从访存开始到把第一个结果元素存入V_4所需的拍数（亦称为**链接流水线的建立时间**）为：

  [（1+6+1）] +[（1+7+1）] = 17 （拍）

- 3条指令的执行时间为：

  [（1+6+1）]+ [（1+7+1）] +（N−1）

  = N+16 （拍）

进行向量链接的要求

保证：无向量寄存器使用冲突和无功能部件使用冲突

- 只有在前一条指令的第一个结果元素送入结果向量寄存器的那一个时钟周期才可以进行链接。
- 当一条向量指令的两个源操作数分别是两条先行指令的结果寄存器时，要求先行的两条指令产生运算结果的时间必须相等，即要求有关功能部件的通过时间相等。
- 要进行链接执行的向量指令的向量长度必须相等，否则无法进行链接。

3. 分段开采技术

如果向量的长度大于向量寄存器的长度，该如何处理呢？

- 当向量的长度大于向量寄存器的长度时，必须把长向量分成长度固定的段，然后循环分段处理，每一次循环只处理一个向量段。
- 这种技术称为分段开采技术。
  - **由系统硬件和软件控制完成，对程序员是透明的。**

例3.4 设*A*和*B*是长度为*N*的向量，考虑在Cray-1向量处理器上实现以下的循环操作：

```
      DO  10 I = 1, N
10    A (I) = 5.0 * B (I)  + C
```

- 当$N \leqslant 64$时，可以用以下指令序列：

$S_1 \leftarrow 5.0$ ；将常数5.0送入标量寄存器S_1

$S_2 \leftarrow 1.0$ ；将常数1.0送入标量寄存器S_2

$VL \leftarrow N$ ；在向量长度寄存器VL中设置向量长度N

$V_0 \leftarrow B$ ；从存储器中将向量B读入向量寄存器V_0

$V_1 \leftarrow S_1 \times V_0$ ；向量B中的每个元素分别和常数S_1相乘

$V_2 \leftarrow S_2 + V_1$ ；向量V_1中的每个元素分别和常数S_2相加

$A \leftarrow V_2$ ；将计算结果从向量寄存器V_2存入存储器的向量A

- 当N ＞64时，就需要进行分段开采。
  - **循环次数K ：**

$$K = \left\lfloor \frac{N}{64} \right\rfloor$$

  - **余数L：**

$$L = N - 64 \times \left\lfloor \frac{N}{64} \right\rfloor$$

$S_1 \leftarrow 5.0$ ；将常数5.0送入标量寄存器S_1

$S_2 \leftarrow 1.0$ ；将常数1.0送入标量寄存器S_2

$VL \leftarrow L$ ；在向量长度寄存器VL中设置向量长度L

$V_0 \leftarrow B$ ；从存储器中将向量B[0..L-1]读入向量寄存器V_0

$V_1 \leftarrow S_1 * V_0$ ；向量B中的每个元素分别和常数S_1相乘

$V_2 \leftarrow S_2 + V_1$ ；向量V_1中的每个元素分别和常数S_2相加

$A \leftarrow V_2$ ；将计算结果从向量寄存器V_2存入存储器的向量A[0..L-1]

处理余数部分，计算L个元素

For (I=0 to K-1) {

$V_0 \leftarrow B$ ；从存储器中将向量B[L+I*64..L+I*64+63]

；读入向量寄存器V_0

$V_1 \leftarrow S_1 * V_0$ ；向量B中的每个元素分别和常数S_1

；相乘；

$V_2 \leftarrow S_2 + V_1$ ；向量V_1中的每个元素分别和常数S_2

；相加

$A \leftarrow V_2$ ；将计算结果V_2存入存储器的向量

； A[L+I*64… L+I*64+63]

}

循环K次，分段处理

4. 采用多处理机系统

许多新型向量处理机系统采用了多处理机系统结构。例如：

- CRAY-2
  - 包含了4个向量处理机
  - 浮点运算速度最高可达1800MFLOPS
- CRAY Y-MP、C90

  最多可包含16个向量处理机

第三章 实 验

实验资料：第3版教材附录A

实验内容：实验一：MIPS指令系统和体系结构

实验二：流水线及其冲突

实验三：指令调度和延迟分支

实验报告：标明实验步骤和内容，手写实验结果。

注意：必须在阿里云平台上实名认证后，在阿里云上完成实验。实验报告中要有阿里云实验完成情况的截图。

第三章　作业

3.1 任选5个

3.3　　3.4

3.11　　3.12

不要用电子版教材，上面的作业不对！！！